vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention

Ramya Prabhu
Microsoft Research
Bengaluru, India

Ajay Nayak*
Indian Institute of Science
Bengaluru, India

Jayashree Mohan
Microsoft Research
Bengaluru, India

Ramchandran Ramjee
Microsoft Research
Bengaluru, India

Ashish Panwar
Microsoft Research
Bengaluru, India

Abstract

PagedAttention is a popular approach for dynamic memory allocation in LLM serving systems. It enables on-demand allocation of GPU memory to mitigate KV cache fragmentation — a phenomenon that crippled the batch size (and consequently throughput) in prior systems. However, in trying to allocate **physical** memory at runtime, PagedAttention ends up changing the **virtual** memory layout of the KV cache from contiguous to non-contiguous. Such a design leads to non-trivial programming and performance overheads.

We present vAttention — an approach that mitigates fragmentation in physical memory while retaining the virtual memory contiguity of the KV cache. We achieve this by decoupling the allocation of virtual and physical memory using CUDA virtual memory management APIs. We also introduce various LLM-specific optimizations to address the limitations of CUDA virtual memory support. Overall, vAttention is a simpler, portable, and performant alternative to PagedAttention: it supports various attention kernels out-of-the-box and improves LLM serving throughput by up to 1.23× compared to the use of PagedAttention-based kernels of FlashAttention-2 and FlashInfer.

CCS Concepts: • **Software and its engineering** → **Maintaining software**; *Virtual memory*; *Software design techniques*; • **General and reference** → **Performance**; • **Computing methodologies** → *Neural networks.*

Keywords: Large language models; KV cache; fragmentation; memory management

ACM Reference Format:
Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramchandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25), March 30–April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3669940.3707256

[*]Contributed to this work as an intern at Microsoft Research India.

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707256

System/library and issues related to PagedAttention
vLLM [51]: Pioneered PagedAttention. Despite being in an actively maintained code repository, vLLM's PagedAttention kernel is up to 2.8× slower than FlashAttention-2 (Table 7). Furthermore, changing block size changes the execution time of the kernel by as much as 1.9× (Figure 3).
FlashAttention-2 [42]: PagedAttention-based prefill kernel is up to 37% slower than the non-paged kernel (Figure 2) while the decode kernel is up to 12% slower. Initial attempts to add paging support failed unit tests [7].
FlashAttention-3 [67] / SDPA in cuDNN-9 [59]: State-of-the-art attention kernels for NVIDIA Hopper architecture did not support PagedAttention when released.
TensorRT-LLM [6]: Serving throughput dropped by more than 10% in a Python front-end [5]. Recommends using the C++ front-end. Even with C++, we observe up to 5% higher latency in some cases with PagedAttention.
FlashInfer [77]: PagedAttention-based prefill kernel is up to 42% slower than the non-paged kernel (Figure 2).

Table 1. The PagedAttention approach requires an application to explicitly manage dynamically allocated physical memory, including re-writing of attention kernels. These examples highlight the complexity, performance and maintenance challenges associated with this approach.

1 Introduction

Large Language Models (LLMs) are being deployed in a wide range of applications e.g., chat bots, search engines and coding assistants [9, 10, 18, 19, 25, 41, 61]. Given the size and scale of modern LLM deployments, optimizing inference has become extremely important [36, 46, 48, 49, 51, 63, 78, 82].

Batching is a powerful technique to boost LLM serving throughput [35, 51, 63, 78]. However, achieving a large batch size requires careful allocation of GPU memory. For each request, the serving framework stores the activations of all

vAttention：在没有 PagedAttention 的情况下为 LLM 提供动态内存管理

Ramya Prabhu
微软研究院印度班加罗尔

Ajay Nayak* 印度科学研究所 印度班加罗尔

Jayashree Mohan
微软研究院印度班加罗尔

Ramchandran Ramjee
微软研究院印度班加罗尔

Ashish Panwar
微软研究院印度班加罗尔

抽象的

PagedAttention 是 LLM 服务系统中动态内存分配的一种流行方法。它支持按需分配 GPU 内存，以减轻 KV 缓存碎片——这种现象会削弱先前系统中的批量大小（进而削弱吞吐量）。然而，在运行时尝试分配物理内存时，PagedAttention 最终会将 KV 缓存的虚拟内存布局从连续更改为非连续。这样的设计会导致不平凡的编程和性能开销。

我们提出了 vAttention——一种减轻物理内存碎片的方法，同时保留 KV 缓存的虚拟内存连续性。我们通过使用 CUDA 虚拟内存管理 API 解耦虚拟内存和物理内存的分配来实现这一目标。我们还引入了各种 LLM 特定的优化来解决 CUDA 虚拟内存支持的限制。总体而言，vAttention 是 PagedAttention 的更简单、可移植且高性能的替代方案：与使用基于 PagedAttention 的 FlashAttention-2 和 FlashInfer 内核相比，它支持开箱即用的各种注意力内核，并将 LLM 服务吞吐量提高了 1.23×。

CCS 概念： • 软件及其工程 → 维护软件；虚拟内存；软件设计技术； • 一般和参考 → 性能； • 计算方法 → 神经网络。

关键词：大语言模型； KV缓存；碎片化；内存管理

ACM参考格式：
Ramya Prabhu、Ajay Nayak、Jayashree Mohan、Ramchandran Ramjee 和 Ashish Panwar。 2025. vAttention：在没有 PagedAttention 的情况下为 LLM 提供动态内存管理。诉讼中

*Contributed to this work as an intern at Microsoft Research India.

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707256

系统/库以及与 PagedAttention 相关的问题
vLLM [51]：开创了 PagedAttention。尽管位于积极维护的代码存储库中，但 vLLM 的 PagedAttention 内核比 FlashAttention-2 慢 2.8×（表 7）。此外，更改块大小会使内核的执行时间改变多达 1.9× (图 3)。
FlashAttention-2 [42]：基于 PagedAttention 的预填充内核比非分页内核（图 2）慢 37%，而解码内核慢 12%。添加分页支持的最初尝试失败了单元测试 [7]。
FlashAttention-3 [67] / cuDNN-9 [59] 中的 SDPA：NVIDIA Hopper 架构的最先进的注意力内核在发布时不支持 PagedAttention。
TensorRT-LLM [6]：Python 前端的服务吞吐量下降了 10% 以上 [5]。建议使用 C++ 前端。即使使用 C++，我们也观察到在某些情况下使用 PagedAttention 时延迟会高出 5%。
FlashInfer [77]：基于 PagedAttention 的预填充内核比非分页内核慢 42%（图 2）。

表 1. PagedAttention 方法需要应用程序显式管理动态分配的物理内存，包括重写注意力内核。这些示例强调了与此方法相关的复杂性、性能和维护挑战。

第 30 届 ACM 编程语言和操作系统架构支持国际会议，第 1 卷 (ASP-LOS ' 25)，2025 年 3 月 30 日至 4 月 3 日，荷兰鹿特丹。 ACM，美国纽约州纽约市，18 页。 https://doi.org/10.1145/3669940.3707256

1 简介

大型语言模型（LLM）正在广泛的应用中部署，例如聊天机器人、搜索引擎和编码助手[9,10,18,19,25,41,61]。考虑到现代大语言模型的规模和规模，优化推理变得极其重要[36,46,48,49,51,63,78,82]。

批处理是一种提高 LLM 服务吞吐量的强大技术 [35,51,63,78]。然而，实现大批量大小需要仔细分配 GPU 内存。对于每个请求，服务框架都会存储所有请求的激活

the tokens processed so far in GPU memory and reuses them for generating subsequent tokens. This is called the KV cache [36, 63, 78] which accounts for a majority of GPU memory usage during inference. Efficiently allocating GPU memory for the KV cache is challenging for two reasons. First, the per-request KV cache grows slowly (one token per iteration), and second, a request's decode length (or its total KV cache size) is not known ahead of time.

Prior systems like Orca [78] and FasterTransformer [15] allocate memory for each request based on the maximum context length supported by the model (e.g., Yi-34B model supports context length of up to 200K [32]). However, the number of decode tokens generated are far less in practice, e.g., the average decode length for the chat-based sharegpt dataset is 415 tokens [35]. Therefore, static memory allocation could create severe internal fragmentation, limiting batch size and serving throughput.

Inspired by demand paging in OS-based virtual memory systems, vLLM introduced PagedAttention [51] that allocates small blocks of GPU memory on demand i.e., when previously allocated blocks are fully utilized and the model continues to generate more tokens. This approach provides a near-perfect solution for mitigating fragmentation and hence, PagedAttention has become the de facto standard for dynamic memory allocation in LLM serving systems, e.g., TensorRT-LLM, HuggingFace TGI, LightLLM [4, 6, 28] etc.

However, we show that PagedAttention faces a fundamental consequence of dynamic memory allocation: *dynamically allocated objects are not guaranteed to be contiguous*. Note that user-level objects are allocated in virtual memory. Therefore, in trying to enable dynamic allocation of **physical** memory, PagedAttention ends up changing the **virtual** memory layout of KV cache from contiguous to non-contiguous. We argue that this approach has several pitfalls (§3). First, it requires rewriting attention kernels, i.e., to enable de-referencing all tokens of the non-contiguous KV cache. Second, it forces developers to implement a memory manager in the serving framework, i.e., to stitch together dynamically allocated virtual memory blocks. Third, it adds runtime overhead in the critical path of both CPU and GPU execution. Table 1 provides empirical evidence and real-world experiences to support these arguments.

The fundamental issue with PagedAttention and prior systems is that they rely on the reservation-based memory allocation method exposed by the GPU runtime. In this method (used by `cudaMalloc`), the runtime allocates both virtual and physical memory on the GPU meaning that *physical memory is allocated even if the corresponding virtual memory is not accessed*. This is in stark contrast to OS-based demand paging [52, 62]. We show that separating the allocation of virtual and physical memory allows for more effective KV cache memory management. To support our claim, we introduce vAttention (§5) — an approach that stores KV cache in contiguous virtual memory without committing physical memory ahead-of-time. vAttention decouples the allocation of virtual and physical memory using the CUDA virtual memory management (VMM) APIs [12].

In building vAttention, we find that using CUDA VMM support for KV cache management poses two key efficiency challenges for an LLM serving system (§6). First, memory allocation using CUDA VMM APIs incurs high latency because each allocation involves a round-trip to the OS kernel. We tackle latency issues with several LLM-specific optimizations such as overlapping memory allocation with compute, opportunistically allocating pages ahead of time, and deferring memory reclamation. Second, CUDA supports memory allocation only at the granularity of large pages, i.e., in multiples of 2MB. Use of large pages can create significant fragmentation. We address this challenge by modifying the open-source CUDA unified virtual memory driver, adding support for smaller 64KB pages. Our evaluation shows that use of 64KB pages has no negative impact on the performance of attention kernels, i.e., we do not find any evidence of TLB thrashing. Together, these optimizations mitigate fragmentation while hiding the latency cost of on demand memory allocation, making vAttention a simpler, portable and performant alternative to PagedAttention.

Overall, we make the following contributions:

- We present vAttention – a memory management approach that retains the virtual contiguity of KV cache while enabling dynamic allocation of physical memory. Our implementation of vAttention in vLLM seamlessly adds dynamic memory allocation support to various unmodified attention kernels.
- We compare vAttention against PagedAttention-based alternatives of vLLM, FlashAttention-2 and FlashInfer on Yi-6B, Llama-3-8B and Yi-34B with 1-2 A100 GPUs. Using FlashAttention-2's non-paged attention kernel, vAttention outperforms vLLM by up to 1.99× in decode throughput. In long-context scenarios, it also improves the end-to-end LLM serving throughput by up to 1.18× and 1.23× over PagedAttention based kernels of FlashAttention-2 and FlashInfer
- We demonstrate the portability benefit of vAttention with the recently launched FlashAttention-3 kernel (FA3 [67]). FA3 is optimized for the NVIDIA Hopper architecture and was not released with PagedAttention support. vAttention supports FA3 out-of-the-box, leading to 1.26 – 1.5× higher throughout over PagedAttention based FlashAttention-2.

2 Background

2.1 Large Language Models

Given an input sequence, an LLM predicts the probability of an output sequence: a sequence is a set of tokens [51]. Each inference request begins with a prefill phase that processes

到目前为止在 GPU 内存中处理的令牌并重用它们来生成后续令牌。这称为 KV 缓存 [36,63,78]，它在推理过程中占据了 GPU 内存使用的大部分。为 KV 缓存有效分配 GPU 内存具有挑战性，原因有两个。首先，每个请求的 KV 缓存增长缓慢（每次迭代一个令牌），其次，请求的解码长度（或其总 KV 缓存大小）无法提前得知。

Orca [78] 和 FasterTransformer [15] 等现有系统根据模型支持的最大上下文长度为每个请求分配内存（例如，Yi-34B 模型支持高达 200K 的上下文长度 [32]）。然而，实际上生成的解码令牌的数量要少得多，例如，基于聊天的 sharegpt 数据集的平均解码长度是 415 个令牌[35]。因此，静态内存分配可能会产生严重的内部碎片，限制批量大小和服务吞吐量。

受到基于操作系统的虚拟内存系统中的需求分页的启发，vLLM 引入了 PagedAttention [51]，它按需分配小块 GPU 内存，即当先前分配的块被充分利用并且模型继续生成更多令牌时。这种方法为减少碎片提供了近乎完美的解决方案，因此，PagedAttention 已成为 LLM 服务系统中动态内存分配的事实标准，例如 TensorRT-LLM、HuggingFace TGI、LightLLM [4,6,28] 等。

然而，我们表明 PagedAttention 面临动态内存分配的一个基本后果：动态分配的对象不能保证是连续的。请注意，用户级对象是在虚拟内存中分配的。因此，在尝试启用物理内存的动态分配时，PagedAttention 最终将 KV 缓存的虚拟内存布局从连续更改为非连续。我们认为这种方法有几个缺陷（§3）。首先，它需要重写注意力内核，即能够取消引用非连续 KV 缓存的所有令牌。其次，它迫使开发人员在服务框架中实现内存管理器，即将动态分配的虚拟内存块拼接在一起。第三，它增加了 CPU 和 GPU 执行关键路径中的运行时开销。表 1 提供了经验证据和现实世界的经验来支持这些论点。

PagedAttention 和之前的系统的根本问题是它们依赖于 GPU 运行时公开的基于预留的内存分配方法。在此方法（由 cudaMalloc 使用）中，运行时在 GPU 上分配虚拟内存和物理内存，这意味着即使未访问相应的虚拟内存，也会分配物理内存。这与基于操作系统的请求分页形成鲜明对比 [52, 62]。我们证明，分离虚拟内存和物理内存的分配可以实现更有效的 KV 缓存内存管理。为了支持我们的主张，我们引入了 vAttention（第 5 节）——一种将 KV 缓存存储在连续虚拟内存中而无需提交物理内存的方法。

提前记忆。 vAttention 使用 CUDA 虚拟内存管理 (VMM) API [12] 解耦虚拟内存和物理内存的分配。

在构建 vAttention 时，我们发现使用 CUDA VMM 支持 KV 缓存管理给 LLM 服务系统带来了两个关键的效率挑战（§6）。首先，使用 CUDA VMM API 进行内存分配会导致高延迟，因为每次分配都涉及到操作系统内核的往返。我们通过一些特定于 LLM 的优化来解决延迟问题，例如将内存分配与计算重叠、提前机会分配页面以及推迟内存回收。其次，CUDA 仅支持大页面粒度（即 2MB 的倍数）的内存分配。使用大页面可能会产生严重的碎片。我们通过修改开源 CUDA 统一虚拟内存驱动程序，添加对较小 64KB 页面的支持来应对这一挑战。我们的评估表明，使用 64KB 页面对注意力内核的性能没有负面影响，即我们没有发现任何 TLB 抖动的证据。这些优化共同减少了碎片，同时隐藏了按需内存分配的延迟成本，使 vAttention 成为 PagedAttention 的更简单、可移植且高性能的替代方案。

总的来说，我们做出了以下贡献：

- 我们提出了 vAttention——一种内存管理方法，它保留 KV 缓存的虚拟连续性，同时实现物理内存的动态分配。我们在 vLLM 中实现的 vAttention 无缝地为各种未修改的注意力内核添加了动态内存分配支持。
- 我们在具有 1-2 个 A100 GPU 的 Yi-6B、Llama-3-8B 和 Yi-34B 上将 vAttention 与基于 PagedAttention 的 vLLM、FlashAttention-2 和 FlashInfer 替代方案进行比较。使用 FlashAttention-2 的非分页注意力内核，vAttention 的解码吞吐量比 vLLM 高出 1.99×。在长上下文场景中，与基于 PagedAttention 的 FlashAttention-2 和 FlashInfer 内核相比，它还将端到端 LLM 服务吞吐量提高了多达 1.18× 和 1.23×
- 我们通过最近推出的 FlashAttention-3 内核（FA3 [67]）展示了 vAttention 的可移植性优势。 FA3 针对 NVIDIA Hopper 架构进行了优化，并且在发布时并未提供 PagedAttention 支持。 vAttention 支持开箱即用的 FA3，与基于 Page-dAttention 的 FlashAttention-2 相比，吞吐量提高了 1.26 – 1.5×。

2 背景

2.1 大型语言模型

给定输入序列，LLM 预测输出序列的概率：序列是一组标记 [51]。每个推理请求都以预填充阶段开始，该阶段处理

Symbol	Definition
N	Number of layers on a particular worker
H	Number of KV heads on a particular worker
D	Dimension of each attention head
P	Number of bytes needed to store one element
B	Maximum batch size
L	Maximum context length supported by the model
L'	Context length of a request seen thus far

Table 2. Notations used in the paper.

prompt tokens in parallel and produces the first output token of the request. Thereafter, the decode phase iteratively processes the output token generated in the previous step and produces the next output token in every iteration [36].

LLMs are built atop one of the variants of the transformer architecture [73]; an LLM consists of multiple transformer blocks. Internally, a transformer block contains two types of operators: position-wise and sequence-wise [78]. The former category includes feed-forward network, layer normalization, activation, embedding layer, output sampling layer, and residual connections whereas *attention* is a sequence-level operator. We primarily focus on attention since it is the primary consumer of GPU memory in LLM inference. Table 2 summarizes the notations used in the paper.

For the attention operator, the model computes the query, key and value vectors from a given sequence of tokens $(x_1, x_2, ..., x_K) \in \mathbb{R}^{K\times E}$ where E represents the embedding size of the model. For each x_i, query, key and value vectors are computed as follows:

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i \tag{1}$$

The resulting k_i and v_i are appended to the key and value vectors of the prior tokens of the corresponding request, producing two matrices $K, V \in \mathbb{R}^{L'\times(H\times D)}$ where L' represents the context length of the request seen so far, H is the number of KV heads of the model on a worker and D is the dimension of each KV head. Then, attention is computed as follows:

$$Attention(q_i, K, V) = softmax(\frac{q_i K^T}{scale})V \tag{2}$$

The attention score is computed separately for each request in the batch. A request executes until the model generates a special end-of-sequence token or reaches the maximum context length for the request. Note that in each iteration of a request, all its preceding k_i and v_i are needed to compute attention. Hence, an inference engine stores the k_i and v_i vectors in memory to reuse them across iterations. We refer to this state of all layers collectively as KV cache. In systems prior to PagedAttention, the K cache (or V cache) at each layer of a worker is typically allocated as a 4D tensor of shape $[B, L, H, D]$ where B refers to batch size and L refers to the maximum possible context length of a request.

2.2 Fragmentation and PagedAttention

Serving LLMs with high throughput requires careful allocation of GPU memory. This is challenging because the total context length of a request is not known in advance. Serving systems worked around this challenge by pre-reserving KV cache space assuming that each context is as long as the maximum length supported by the model (e.g., 200K for Yi-34B-200K). vLLM shows that this strategy is prone to severe internal fragmentation. In fact, vLLM showed that prior reservation is suboptimal even if the context lengths are known in advance. This is because the per-request KV cache grows one token at a time and hence prior reservation wastes memory over the entire lifetime of a request.

Inspired by the OS-based virtual memory systems, vLLM proposed PagedAttention to mitigate fragmentation by dynamically allocating memory for the KV cache. PagedAttention splits KV cache into fixed-sized blocks and allocates memory for one block at a time. This way, vLLM allocates only as much memory as a request needs, and only when required – not ahead-of-time.

GPUs and page sizes: NVIDIA GPUs support multiple page sizes in the hardware [22, 58, 65, 79]. A single call to a CUDA VMM API can allocate one or more physical pages, which we refer to as a page-group. We use page-groups to support multiple allocation granularities for the KV cache, similar to how multiple page sizes are commonly used in conventional OS-based virtual memory systems [52, 62].

3 Issues with the PagedAttention Approach

Despite being inspired by demand paging, the PagedAttention approach is different from it: *PagedAttention implements demand paging in user space whereas conventional demand paging is transparent to applications.* This section elaborates on issues that arise with such an approach.

3.1 Requires Re-writing the Attention Kernel

Conventional implementations of the attention operator assume that the two input tensors K and V (Equation 2) are stored in contiguous memory. By departing from the conventional memory layout, PagedAttention requires an implementation of the attention operator to be modified so as to compute attention scores over non-contiguous KV cache blocks. Writing correct and performant GPU kernels can be challenging for most programmers [7].

Being a fundamental building block of the transformer architecture, the attention operator has witnessed a tremendous pace of innovation in the systems and ML communities for performance optimizations [2, 37–40, 42, 43, 47, 49, 68, 77, 80], and this trend is likely to continue. In the PagedAttention model, keeping up with new research requires continued efforts in porting new optimizations to a PagedAttention-aware implementation. Production systems can therefore easily fall behind research, potentially losing performance

Symbol	Definition
N	Number of layers on a particular worker
H	Number of KV heads on a particular worker
D	Dimension of each attention head
P	Number of bytes needed to store one element
B	Maximum batch size
L	Maximum context length supported by the model
L'	Context length of a request seen thus far

表 2. 论文中使用的符号。

并行提示令牌并生成请求的第一个输出令牌。此后，解码阶段迭代地处理上一步中生成的输出令牌，并在每次迭代中产生下一个输出令牌[36]。

LLM 构建在 Transformer 架构的一种变体之上 [73]; 大语言模型个变压器模块组成。在内部，变压器块包含两种类型的运算符：位置式和序列式[78]。前一类包括前馈网络、层归一化、激活、嵌入层、输出采样层和残差连接，而注意力是序列级算子。我们主要关注注意力，因为它是 LLM 推理中 GPU 内存的主要消耗者。表 2 总结了本文中使用的符号。

对于注意力运算符，模型根据给定的标记序列 $(x_1, x_2, ..., x_K) \in \mathbb{R}^{K\times E}$ 计算查询、键和值向量，其中 E 表示模型的嵌入大小。对于每个 x_i，查询、键和值向量计算如下：

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i \tag{1}$$

得到的 k_i 和 v_i 被附加到相应请求的先前令牌的键和值向量中，产生两个矩阵 $K, V \in \mathbb{R}^{L'\times(H\times D)}$，其中 L' 表示到目前为止看到的请求的上下文长度，H 是工作器上模型的 KV 头的数量，D 是每个 KV 头的维度。然后，注意力计算如下：

$$Attention(q_i, K, V) = softmax(\frac{q_i K^T}{scale})V \tag{2}$$

批次中每个请求的注意力分数是单独计算的。请求将一直执行，直到模型生成特殊的序列结束标记或达到请求的最大上下文长度。请注意，在请求的每次迭代中，需要其前面的所有 k_i 和 v_i 来计算注意力。因此，推理引擎将 k_i 和 v_i 向量存储在内存中，以便在迭代中重用它们。我们将所有层的这种状态统称为 KV 缓存。在 PagedAttention 之前的系统中，工作线程每一层的 K 缓存（或 V 缓存）通常被分配为形状为 $[B, L, H, D]$ 的 4D 张量，其中 B 指批量大小，L 指请求的最大可能上下文长度。

2.2 分片和PagedAttention

为 LLM 提供高吞吐量服务需要仔细分配 GPU 内存。这是具有挑战性的，因为请求的总上下文长度是事先未知的。服务系统通过预先保留 KV 缓存空间来解决这一挑战，假设每个上下文都与模型支持的最大长度一样长（例如，Yi-34B-200K 为 200K）。vLLM 表明这种策略容易出现严重的内部碎片。事实上，vLLM 表明，即使提前知道上下文长度，事先预留也不是最优的。这是因为每个请求的 KV 缓存一次会增加一个令牌，因此先前的预留会在请求的整个生命周期内浪费内存。

受到基于操作系统的虚拟内存系统的启发，vLLM 提出了 PagedAttention，通过为 KV 缓存动态分配内存来减少碎片。 PagedAttention 将 KV 缓存分割成固定大小的块，并一次为一个块分配内存。这样，vLLM 仅在需要时分配请求所需的内存，而不是提前分配。

GPU 和页面大小：NVIDIA GPU 在硬件中支持多种页面大小 [22、58、65、79]。对 CUDA VMM API 的一次调用可以分配一个或多个物理页面，我们将其称为页面组。我们使用页面组来支持 KV 缓存的多个分配粒度，类似于传统的基于操作系统的虚拟内存系统中通常使用多个页面大小的方式 [52, 62]。

3 PagedAttention 方法的问题

尽管受到需求分页的启发，但 PagedAttention 方法与之不同：PagedAttention 在用户空间中实现需求分页，而传统的需求分页对应用程序是透明的。本节详细阐述了这种方法所出现的问题。

3.1 需要重写Attention Kernel

注意力算子的传统实现假设两个输入张量 K 和 V（方程 2）存储在连续的内存中。与传统的内存布局不同，PagedAttention 需要修改注意力运算符的实现，以便计算非连续 KV 缓存块上的注意力分数。对于大多数程序员来说，编写正确且高性能的 GPU 内核可能具有挑战性 [7]。

作为 Transformer 架构的基本构建模块，注意力算子见证了系统和 ML 社区在性能优化方面的巨大创新步伐 [2, 37–40, 42, 43, 47, 49, 68, 77, 80]，并且这种趋势可能会持续下去。在 PagedAttention 模型中，跟上新的研究需要不断努力将新的优化移植到 PagedAttention 感知的实现中。因此，生产系统很容易落后于研究，可能会损失性能

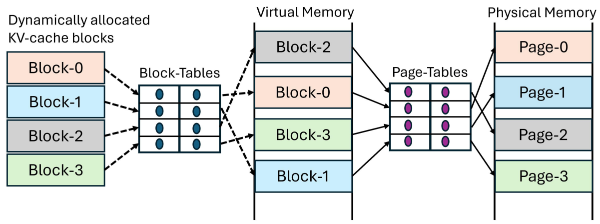


Figure 1. PagedAttention involves two layers of memory management: one in user space and one in OS kernel space.

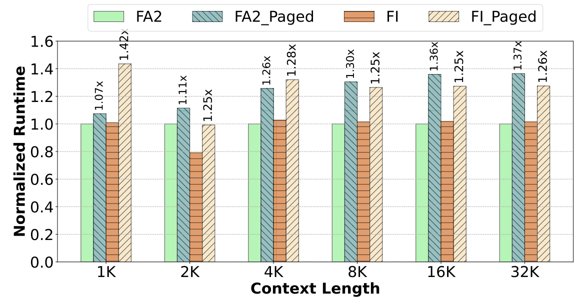


Figure 2. Overhead of PagedAttention in prefill kernels (model: Llama-3-8B, one A100 GPU). Numbers on top show overhead over the corresponding non-paged implementation of FlashAttention-2 (FA2) and FlashInfer (FI).

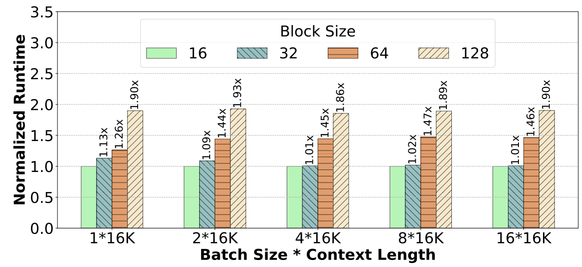


Figure 3. Latency of vLLM's paged decode kernel is sensitive to block size (model: Llama-3-8B, one A100 GPU).

and competitive advantage. To provide an example, Table 7 shows that the paged kernel of vLLM is already up to 2.8× slower than the FlashAttention-2 kernel [37].

3.2 Adds Redundancy in the Serving Framework

PagedAttention makes an LLM serving system responsible for managing the mappings between KV cache and dynamically allocated memory blocks. For example, consider a request that allocates four KV cache blocks over time (left half of Figure 1). These blocks are usually non-contiguous in virtual memory. During the computation of Equation 2, PagedAttention kernel needs to access all the elements of the four KV cache blocks. To facilitate this, the serving system needs to track the virtual memory addresses of KV cache blocks and pass them to the attention kernel at runtime. This approach effectively requires duplicating what the operating system already does for enabling virtual-to-physical address translation (right half in Figure 1).

3.3 Performance Overhead

3.3.1 Runtime overhead on the GPU. PagedAttention slows down attention computation by adding extra code in the critical path of execution. For example, the vLLM paper acknowledges that the PagedAttention-based implementation was $20-26\%$ slower than the corresponding none-paged FasterTransformer kernel, primarily due to the overhead of looking up Block-Tables and executing extra branches (see Figure 18a in [51]). In addition, Figure 2 shows that incorporating PagedAttention has also added a significant performance overhead in other state-of-the-art kernel libraries. For example, PagedAttention based prefill kernels of FlashAttention-2 and FlashInfer are up to 37% and 42% slower than the non-paged kernels in the corresponding libraries. Our analysis reveals that the number of instructions executed in PagedAttention kernels is $7-13\%$ higher than the non-paged kernels. Caching page indices also increases register pressure, causing register spilling.

To highlight another example of difficulty involved in writing an efficient paged kernel, Figure 3 shows that the performance of vLLM's paged decode kernel is significantly worse with large block sizes of 64 and 128. Our analysis indicates that this is likely due to L1 cache efficiency: smaller blocks have a higher memory bandwidth utilization due to higher hit rates in the L1 cache.

3.3.2 Runtime overhead on the CPU. Implementing an additional memory manager can add performance issues in the CPU runtime of the serving system. We refer to a few real-world examples and our own observations on vLLM to corroborate this argument.

To enable PagedAttention, a serving system needs to supply Block-Tables to the attention kernel. In vLLM, the latency of preparing a Block-Table depends on batch composition and grows proportional to `max_num_blocks` × `batch_size` where `max_num_blocks` refers to the number of KV cache blocks in the longest request of the batch. This is because vLLM manages a Block-Table as a 2D tensor and aligns the number of KV cache blocks in each request by padding unoccupied slots with zeros. If a batch contains a few long and many short requests, such padding results in a significant overhead. In our earlier experiments, we observed that Block-Table preparation in vLLM was contributing 30% latency in decode iterations. While a recent fix [17] has mitigated some of this overhead, we find that it can still be as high as 10%. High overhead of PagedAttention has also been found in TensorRT-LLM, degrading throughput by 11% [5]. This issue was attributed to the Python runtime of TensorRT-LLM and

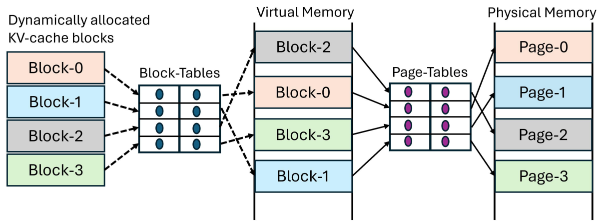


图 1. PagedAttention 涉及两层内存管理：一层位于用户空间，一层位于操作系统内核空间。

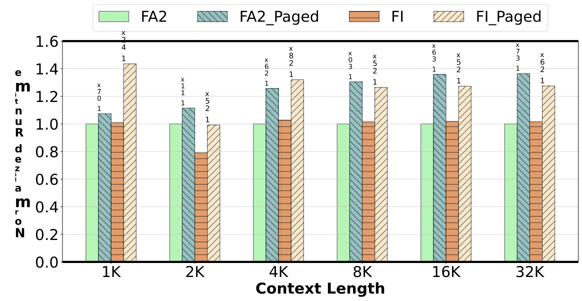


图 2. 预填充内核中 PagedAttention 的开销（型号：Llama-3-8B，一个 A100 GPU）。顶部的数字显示了 FlashAttention-2 (FA2) 和 FlashInfer (FI) 的相应非分页实现的开销。

和竞争优势。举个例子，表 7 显示 vLLM 的分页内核已经比 FlashAttention-2 内核慢 2.8× [37]。

3.2 在服务框架中添加冗余

PagedAttention 使 LLM 服务系统负责管理 KV 缓存和动态分配的内存块之间的映射。例如，考虑一个随时间分配四个 KV 缓存块的请求（图 1 的左半部分）。这些块在虚拟内存中通常是不连续的。在公式2的计算过程中，PagedAttention内核需要访问四个KV缓存块的所有元素。为了实现这一点，服务系统需要跟踪 KV 缓存块的虚拟内存地址，并在运行时将它们传递给注意力内核。这种方法实际上需要重复操作系统已经执行的用于实现虚拟到物理地址转换的操作（图 1 中的右半部分）。

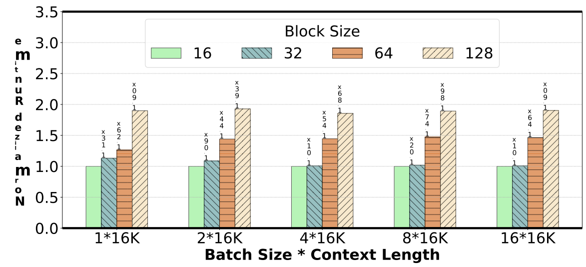


图 3. vLLM 分页解码内核的延迟对块大小敏感（型号：Llama-3-8B，一个 A100 GPU）。

3.3 性能开销

3.3.1 GPU 上的运行时开销。 PagedAttention 通过在关键执行路径中添加额外的代码来减慢注意力计算速度。例如，vLLM 论文承认基于 PagedAttention 的实现比相应的非分页 FasterTransformer 内核慢 20–26%，这主要是由于查找块表和执行额外分支的开销（参见[51]中的图 18a）。此外，图 2 显示，在其他最先进的内核库中，合并 PagedAttention 也增加了显着的性能开销。例如，FlashAttention-2 和 FlashInfer 的基于 PagedAttention 的预填充内核比相应库中的非分页内核慢了 37% 和 42%。我们的分析表明，PagedAttention 内核中执行的指令数量比非分页内核高 7 – 13%。缓存页索引还会增加寄存器压力，导致寄存器溢出。

为了强调编写高效分页内核所涉及的困难的另一个示例，图 3 显示 vLLM 的分页解码内核的性能在 64 和 128 的大块大小下显着变差。我们的分析表明，这可能是由于 L1 缓存效率所致：较小

由于 L1 缓存中的命中率较高，块具有较高的内存带宽利用率。

3.3.2 CPU 上的运行时开销。实施额外的内存管理器可能会增加服务系统的 CPU 运行时的性能问题。我们参考了一些现实世界的例子以及我们自己对 vLLM 的观察来证实这一论点。

为了启用 PagedAttention，服务系统需要向注意力内核提供块表。在 vLLM 中，准备块表的延迟取决于批次组成，并且与 max_num_blocks × batch_size 成正比增长，其中 max_num_blocks 指批次中最长请求中的 KV 缓存块的数量。这是因为 vLLM 将块表作为 2D 张量进行管理，并通过用零填充未占用的槽来对齐每个请求中的 KV 缓存块的数量。如果一个批次包含一些长请求和许多短请求，则这种填充会导致显着的开销。在我们早期的实验中，我们观察到 vLLM 中的块表准备在解码迭代中造成了 30% 的延迟。虽然最近的修复 [17] 减轻了部分开销，但我们发现它仍然可能高达 10%。 TensorRT-LLM 中也发现 PagedAttention 的高开销，导致吞吐量下降 11% [5]。此问题归因于 TensorRT-LLM 的 Python 运行时和

moving to a C++ runtime can mitigate the CPU overhead. However, doing so requires non-trivial programming effort.

Overall, this section shows that PagedAttention adds a significant programming burden while also being inefficient. In vAttention, we propose a more principled approach to dynamic KV cache memory management. However, before delving into vAttention, we first highlight some of the fundamental characteristics of LLM serving workloads from a memory management perspective.

4 Insights into LLM Serving Systems

To understand the memory allocation pattern of LLM serving systems, we experiment with Yi-6B running on a single A100 GPU, and Llama-3-8B and Yi-34B running on two A100 GPUs with tensor-parallelism (TP). We set the initial context length of each request to 1K tokens, vary the batch size from 1 to 320 and measure the throughput and memory requirement of the decode phase (see §6 for our discussion and optimizations for the prefill phase).

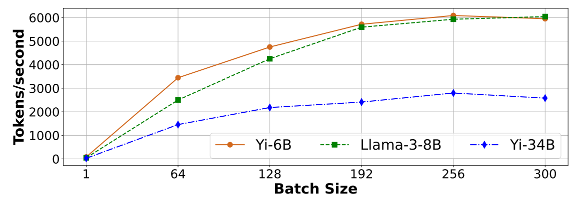
Observation-1: *KV cache memory requirement is predictable on a per-iteration basis.* Due to auto-regressive decoding, once a request enters the decode phase, its KV cache size increases uniformly by one token per iteration. This allows the serving system to determine in advance if additional memory will be required during the iteration's execution.[1]

Observation-2: *KV cache does not require high memory allocation bandwidth.* The memory footprint of a single token across all layers is typically few 10s-100s of kilobytes of memory. For example, the per-token memory footprint of Yi-6B, Llama-3-8B and Yi-34B is 64KB, 128KB and 240KB, respectively. Further, each iteration runs for 10s-100s of milliseconds implying that a request requires at most a few megabytes of memory per second. While batching improves system throughput [35, 36, 63, 82], the number of tokens generated per second plateaus beyond a certain batch size (Figure 4a). This implies that the memory allocation bandwidth requirement also saturates at large batch sizes (e.g., at 256 for Yi-34B). For all the models we studied, we observe that the highest memory allocation rate is at most 750MB per second (Figure 4b). vAttention leverages these observations to optimize KV cache memory management.

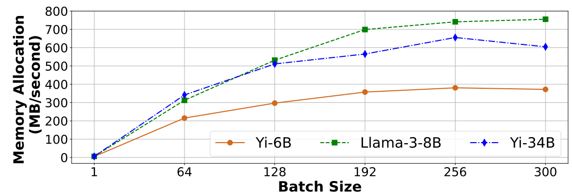
5 vAttention: Design and Implementation

Our primary observation is that *physical memory fragmentation can be avoided without making KV cache non-contiguous in virtual memory.* To realize this, vAttention decouples the allocation of virtual memory from physical memory by leveraging system support for demand paging (instead of implementing demand paging in user space, as in PagedAttention).

[1]Note that this prediction is possible only at the granularity of individual iterations; the total KV cache requirement of a request remains unknown as it depends on the total number of output tokens in the request.



(a) Decode throughput.



(b) Rate of memory allocation.

Figure 4. Decode throughput (top) and the rate of physical memory allocation (bottom) saturate at large batch sizes.

5.1 Design Overview

vAttention employs distinct allocation policies for virtual and physical memory. Specifically, we allocate a large contiguous buffer for the KV cache in virtual memory ahead-of-time (similar to systems prior to PagedAttention) while deferring the allocation of physical memory to runtime (similar to PagedAttention). This design preserves virtual contiguity of KV cache without fragmenting physical memory. Note that this approach could fragment and waste virtual memory. However, this is not an issue since virtual memory is abundant, e.g., modern 64-bit systems provide a 128TB user-addressable virtual memory per process.[2]

5.1.1 Pre-reserving virtual memory. Since virtual memory is abundant, we pre-allocate it in size that is large enough to hold the KV cache of the maximum batch size (configurable) that needs to be supported. In doing so, we assume that each request's context length is same as the maximum supported by the model.

5.1.2 Number of virtual memory buffers. A serving framework maintains separate K and V tensors for each layer of the model. Therefore, we reserve $2 \times N$ buffers on a worker where N is the number of layers managed by that worker.

5.1.3 Size of a virtual memory buffer. The maximum size of a buffer is $BS = B \times S$ where B is the maximum batch size and S is the maximum size of a single request's per-layer K cache (or V cache) on a worker. Further, $S = L \times H \times D \times P$, where L is the maximum context length supported by the model, H is the number of KV heads on a worker, D is the dimension of each KV head and P is the

[2]64-bit systems use only 48 bits for virtual addresses today, providing a per-process virtual memory space of 256TB which is divided equally between the user space and (OS) kernel space.

迁移到 C++ 运行时可以减轻 CPU 开销。然而，这样做需要大量的编程工作。

总的来说，本节表明 PagedAttention 增加了显着的编程负担，同时效率也很低。在 vAttention 中，我们提出了一种更有原则的动态 KV 缓存管理方法。然而，在深入研究 vAttention 之前，我们首先从内存管理的角度强调 LLM 服务工作负载的一些基本特征。

4 深入了解 LLM 服务系统

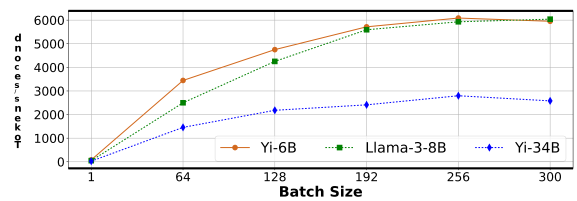
为了了解 LLM 服务系统的内存分配模式，我们实验了在单个 A100 GPU 上运行的 Yi-6B，以及在两个具有张量并行性 (TP) 的 A100 GPU 上运行的 Llama-3-8B 和 Yi -34B。我们将每个请求的初始上下文长度设置为 1K 令牌，将批量大小从 1 更改为 320，并测量解码阶段的吞吐量和内存要求（有关预填充阶段的讨论和优化，请参阅第 6 节）。

观察 1：KV 缓存内存需求在每次迭代的基础上是可预测的。由于自回归解码，一旦请求进入解码阶段，其 KV 缓存大小每次迭代都会均匀增加 1 个令牌。这使得服务系统可以提前确定迭代执行期间是否需要额外的内存。[1]

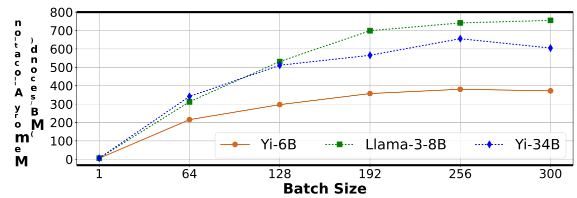
观察2：KV缓存不需要高内存分配带宽。跨所有层的单个令牌的内存占用通常是几十到几百千字节的内存。例如，Yi-6B、Llama-3-8B 和 Yi-34B 的每个令牌内存占用量分别为 64KB、128KB 和 240KB。此外，每次迭代运行 10 到 100 毫秒，这意味着请求每秒最多需要几兆字节的内存。虽然批处理提高了系统吞吐量 [35,36,63,82]，但每秒生成的令牌数量在超过特定批处理大小后会趋于稳定（图 4a）。这意味着内存分配带宽要求在大批量大小时也会饱和（例如，Yi-34B 为 256）。对于我们研究的所有模型，我们观察到最高内存分配率最多为每秒 750MB（图 4b）。 vAttention 利用这些观察结果来优化 KV 缓存管理。

5 vAttention：设计与实现

我们的主要观察是，无需使 KV 缓存在虚拟内存中不连续，就可以避免物理内存碎片。为了实现这一点，vAttention 通过利用系统对需求分页的支持（而不是像 PagedAttention 那样在用户空间中实现需求分页），将虚拟内存的分配与物理内存分离。



(a) Decode throughput.



(b) Rate of memory allocation.

图 4. 解码吞吐量（顶部）和物理内存分配率（底部）在大批量大小时饱和。

5.1 设计概述

vAttention 对虚拟内存和物理内存采用不同的分配策略。具体来说，我们提前在虚拟内存中为 KV 缓存分配一个大的连续缓冲区（类似于 PagedAttention 之前的系统），同时将物理内存的分配推迟到运行时（类似于 PagedAttention）。这种设计保留了 KV 缓存的虚拟连续性，而不会造成物理内存碎片。请注意，这种方法可能会产生碎片并浪费虚拟内存。然而，这不是问题，因为虚拟内存非常丰富，例如，现代 64 位系统为每个进程提供 128TB 用户可寻址虚拟内存。[2]

5.1.1 预先保留虚拟内存。由于虚拟内存充足，我们预先分配足够大的大小来容纳需要支持的最大批量大小（可配置）的KV缓存。在此过程中，我们假设每个请求的上下文长度与模型支持的最大长度相同。

5.1.2 虚拟内存缓冲区的数量。服务框架为模型的每一层维护单独的 K 和 V 张量。因此，我们在一个工作线程上保留 $2 \times N$ 个缓冲区，其中 N 是该工作线程管理的层数。

5.1.3 虚拟内存缓冲区的大小。缓冲区的最大大小为 $BS = B \times S$，其中 B 是最大批量大小，S 是工作线程上单个请求的每层 K 缓存（或 V 缓存）的最大大小。此外，$S = L \times H \times D \times P$，其中$L$是模型支持的最大上下文长度，$H$是worker上的KV头的数量，$D$是每个KV头的维度，$P$是

[1] Note that this prediction is possible only at the granularity of individual iterations; the total KV cache requirement of a request remains unknown as it depends on the total number of output tokens in the request.

[2] 64-bit systems use only 48 bits for virtual addresses today, providing a per-process virtual memory space of 256TB which is divided equally between the user space and (OS) kernel space.

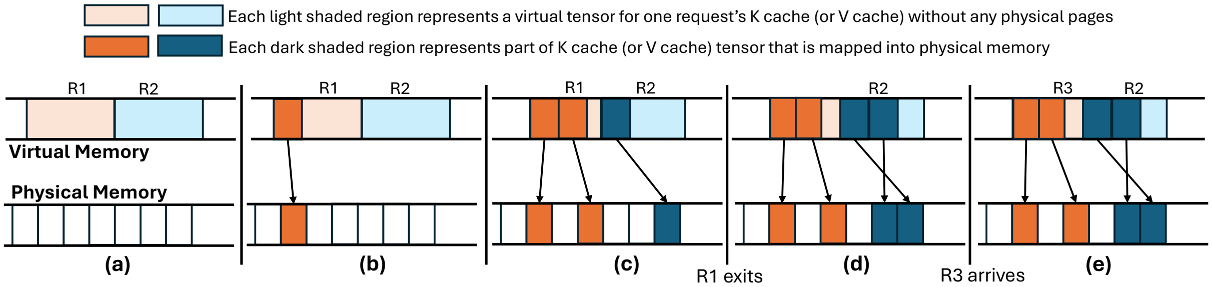


Figure 5. Dynamic memory management in vAttention for a single K cache (or V cache) tensor. (a) shows a virtual tensor for a batch of two requests with no physical memory allocation yet. (b) R1 is allocated one physical page. (c) R1 is allocated two pages and R2 is allocated one page. (d) R1 has completed but vAttention does not reclaim its memory (deferred reclamation). (e) when R3 arrives, vAttention assigns R1's tensor to it which is already backed by physical memory.

CUDA VM APIs	vAttention VM APIs	Description	Latency (microseconds) 64KB	128KB	256KB	2MB
`cuMemAddressReserve` *	`vMemReserve` *	Allocate a buffer in virtual memory	18	17	16	2
`cuMemCreate` *	`vMemCreate` *	Allocate a handle in physical memory	1.7	2	2.1	29
`cuMemMap`	`vMemMap`	Map a physical handle to a virtual buffer	8	8.5	9	2
`cuMemSetAccess`	-	Enable access to a virtual buffer	-	-	-	38
`cuMemUnmap`	-	Unmap physical handle from a virtual buffer	-	-	-	34
`cuMemRelease` *	`vMemRelease` *	Free physical pages of a handle	2	3	4	23
`cuMemAddressFree` *	`vMemFree` *	Free a virtual memory buffer	35	35	35	1

Table 3. CUDA VMM APIs. * represents APIs that we use once while instantiating or terminating the serving framework. Rest of the APIs are used for (un)mapping physical memory pages at runtime. CUDA APIs (prefixed with cu) support only 2MB pages, whereas our CUDA extension APIs (prefixed with v) support fine-grained allocations.

number of bytes based on model precision (e.g., P=2 for FP16/BF16). As an example, consider Yi-34B with FP16 and two-way tensor-parallelism (TP-2). In this case, $N=60, H=4, D=128, P=2$ (8 KV heads of Yi-34B are split evenly on two GPUs), and maximum supported context length $L=200K$. For this configuration, $S=200MB$ $(200K*4*128*2)$. Assuming $B=500$, the maximum size of each buffer per-worker is $BS=100GB$ $(500\times 200MB)$. Therefore, the total virtual memory requirement for 60 layers is 120 buffers of 100GB each (12TB total). Note that the size of virtual address space available grows with the number of workers, e.g., with two TP workers, the total size of user-addressable virtual address space is 256TB. Therefore, virtual memory is always plentiful to satisfy large allocations. Figure 5 shows an example of how vAttention allocates physical memory pages dynamically.

5.2 Leveraging CUDA Virtual Memory Support

The standard GPU memory allocation interface `cudaMalloc` does not support demand paging, i.e., it allocates virtual memory and physical memory at the same time. However, recent CUDA versions provide programmers a fine-grained control over managing virtual and physical memory, including support for decoupling their allocations [12, 45]. We leverage these low-level APIs.

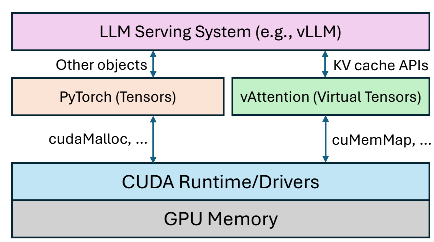


Figure 6. An LLM serving system interacts with vAttention for KV cache management with a set of simple APIs listed in Table 4. All other memory objects (e.g., activations) are allocated by the standard PyTorch caching allocator.

5.2.1 CUDA virtual memory APIs.

Table 3 provides an overview of CUDA VMM APIs that allow decoupling the allocation of virtual memory from physical memory. The allocation granularity depends on the page size used by the GPU. Further, the size of a virtual memory buffer or a physical memory handle must be a multiple of the physical memory allocation granularity. Physical memory pages can be allocated to (or de-allocated from) sub-regions in a virtual memory buffer independently of other sub-regions.

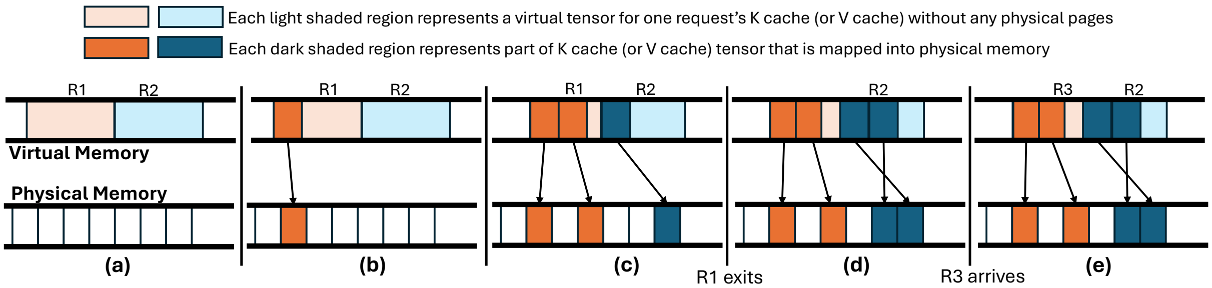


图 5. vAttention 中针对单个 K 缓存（或 V 缓存）张量的动态内存管理。 (a) 显示了一批两个请求的虚拟张量，尚未分配物理内存。 (b) R1 被分配一个物理页。 (c) R1 被分配两页，R2 被分配一页。 (d) R1 已完成，但 vAttention 未回收其内存（延迟回收）。 (e) 当 R3 到达时，vAttention 将 R1 的张量分配给它，该张量已经由物理内存支持。

CUDA VM API	vAttention VM API	描述	延迟（微秒）64KB	128KB	256KB	2MB
cuMemAddressReserve *	vMemReserve *	在虚拟内存中分配缓冲区	18	17	16	2
cuMemCreate *	vMemCreate *	在物理内存中分配句柄	1.7	2	2.1	29
cuMemMap	vMemMap	将物理句柄映射到虚拟缓冲区	8	8.5	9	2
cuMemSetAccess	-	启用对虚拟缓冲区的访问	-	-	-	38
cuMemUnmap	-	从虚拟缓冲区取消映射物理句柄	-	-	-	34
cuMemRelease *	vMemRelease *	释放句柄的物理页	2	3	4	23
cuMemAddressFree *	vMemFree *	释放虚拟内存缓冲区	35	35	35	1

表 3. CUDA VMM API。 * 代表我们在实例化或终止服务框架时使用一次的 API。其余 API 用于在运行时映射（取消）物理内存页。 CUDA API（以 cu 为前缀）仅支持 2MB 页面，而我们的 CUDA 扩展 API（以 v 为前缀）支持细粒度分配。

基于模型精度的字节数（例如，FP16/BF16 的 P=2）。例如，考虑具有 FP16 和双向张量并行性 (TP-2) 的 Yi-34B。在本例中，$N = 60, H = 4, D = 128, P = 2$（Yi-34B 的 8 KV 磁头均匀分配在两个 GPU 上），最大支持的上下文长度为 $L = 200K$。对于此配置，$S = 200MB$ ($200K * 4 * 128 * 2$)。假设 $B = 500$，每个工作线程的每个缓冲区的最大大小为 $BS = 100GB$ ($500 \times 200MB$)。因此，60 层的总虚拟内存需求为 120 个缓冲区，每个缓冲区 100GB（总共 12TB）。请注意，可用虚拟地址空间的大小随着工作线程数量的增加而增长，例如，有两个 TP 工作线程，用户可寻址虚拟地址空间的总大小为 256TB。因此，虚拟内存总是充足的，可以满足大量分配。图 5 显示了 vAttention 如何动态分配物理内存页面的示例。

5.2 利用 CUDA 虚拟内存支持

标准GPU内存分配接口cudaMalloc不支持按需分页，即同时分配虚拟内存和物理内存。然而，最近的 CUDA 版本为程序员提供了管理虚拟和物理内存的细粒度控制，包括支持解耦其分配 [12, 45]。我们利用这些低级 API。

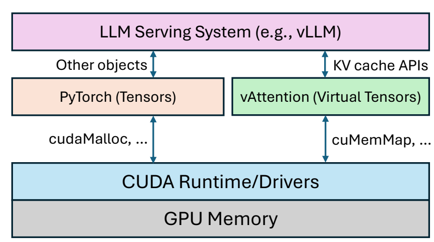


图 6.LLM 服务系统通过表 4 中列出的一组简单 API 与 vAttention 交互以进行 KV 缓存管理。所有其他内存对象（例如激活）均由标准 PyTorch 缓存分配器分配。

5.2.1 CUDA 虚拟内存 API。

表 3 概述了 CUDA VMM API，这些 API 允许将虚拟内存的分配与物理内存分离。分配粒度取决于 GPU 使用的页面大小。此外，虚拟内存缓冲区或物理内存句柄的大小必须是物理内存分配粒度的倍数。物理内存页可以独立于其他子区域被分配到虚拟内存缓冲区中的子区域（或从子区域解除分配）。

APIs	Description
`init`	Initializes vAttention with model parameters. arguments: N, B, L, H, D, P, `page-group-size`. return value: a list of KV cache tensors.
`alloc_reqid`	Allocates an unused `reqId` and marks it active arguments: None return value: an integer `reqId`
`free_reqid`	Frees a `reqId` and marks it inactive arguments: an integer `reqId` return value: None
`step`	Ensures KV cache tensors are backed by physical pages up to the current context length of each active request arguments: an array of size B containing sequence lengths of each `reqId` return value: 0 (success), -1 (failure).

Table 4. Key APIs that vAttention exposes to a serving framework for dynamic KV cache memory management.

5.2.2 Extending PyTorch caching allocator. KV cache is a collection of tensors. In current deep learning frameworks such as PyTorch, a tensor allocated via APIs such as `torch.empty` comes with pre-allocated physical memory. This is because the PyTorch caching allocator relies on the `cudaMalloc` interface (Figure 6). Relying on the low-level API support from CUDA, we extend the PyTorch caching allocator to allow an application to reserve a virtual memory buffer for a tensor without committing physical memory ahead-of-time. We refer to tensors allocated via these APIs as virtual tensors.

5.2.3 Request-level KV cache indexing. A virtual tensor represents the K cache (or V cache) of a layer for the maximum batch size B. In these tensors, different requests occupy different non-overlapping sub-regions (say sub-tensors). We locate the sub-tensor of a request with a unique integer identifier `reqId` that lies in the range of 0 to $B - 1$ (note that at most B requests run simultaneously). The K cache (or V cache) offset of a request's sub-tensor in the virtual tensor of the entire batch is `reqId` $\times S$ where S is the maximum size of per-layer K cache (or V cache) of a request on a worker. The request identifier `reqId` is allocated by vAttention.

5.3 Serving LLMs with vAttention

We build vAttention as a Python library that internally uses a CUDA/C++ extension for interacting with CUDA drivers. Our library exposes a set of simple APIs to the serving framework (shown in Figure 6, Table 4 and Algorithm 1). For simplicity, we discuss the use of these APIs from a single worker's perspective; all workers behave the same.

5.3.1 Initial setup. When the serving framework starts, each model worker loads the vAttention library and configures it with model parameters N, H, D, P, B and a preferred *page-group size* (§6.2) via the `init` API (line 4 in Algorithm 1).

Algorithm 1 Using vAttention in a serving framework.

```
1:  max_batch_size ← B
2:  cache_seq_len ← [0]*B
3:  req_batch_idx ← dict()
4:  vattention.init(config_params)
5:  while !request_pool.is_empty() do
6:      for R_i in new_requests do
7:          if can_schedule(R_i) then
8:              idx ← vattention.alloc_reqid()
9:              req_batch_idx[R_i] ← idx
10:             cache_seq_len[idx] ← prompt_len(R_i)
11:         end if
12:     end for
13:     vattention.step(cache_seq_len)
14:     model.forward()
15:     for R_i in active_requests do
16:         idx ← req_batch_idx[R_i]
17:         if is_complete(R_i) then
18:             cache_seq_len[idx] ← 0
19:             vattention.free_reqid(idx)
20:         else
21:             cache_seq_len[idx] + = 1
22:         end if
23:     end for
24: end while
```

Internally, vAttention reserves $2 \times N$ virtual tensors on the worker, as shown in Figure 5(a), where N is the number of layers hosted by the worker. These virtual tensors are reserved for the lifetime of the serving application. In addition, vAttention also pre-allocates physical memory pages at each worker during initialization. However, these pages are not mapped into the KV cache at this point.

5.3.2 Scheduling a new request. When a new request is scheduled for the first time, the serving framework obtains a new `reqId` from vAttention via `alloc_reqid` (line 8). All subsequent memory management operations of the request are tagged with this `reqId`.

5.3.3 Model execution. Before scheduling a batch for execution, the framework needs to ensure that the KV cache sub-tensors of each active request are backed by physical memory (Figure 5(b) and (c)). For this purpose, before dispatching the first kernel of an iteration to the GPU, the framework invokes the `step` API (line 13), specifying the current context length of each request (context length is set to 0 for each inactive `reqId`). Internally, vAttention ensures that enough physical pages are mapped for each active `reqId` before returning execution back to the framework. If vAttention cannot satisfy the memory demand, it returns with a failure in response to which a serving framework can preempt one or more requests to allow forward progress

APIs	Description
`init`	Initializes vAttention with model parameters. arguments: N, B, L, H, D, P, `page-group-size`. return value: a list of KV cache tensors.
`alloc_reqid`	Allocates an unused `reqId` and marks it active arguments: None return value: an integer `reqId`
`free_reqid`	Frees a `reqId` and marks it inactive arguments: an integer `reqId` return value: None
`step`	Ensures KV cache tensors are backed by physical pages up to the current context length of each active request arguments: an array of size B containing sequence lengths of each `reqId` return value: 0 (success), -1 (failure).

表 4. vAttention 向服务框架公开的用于动态 KV 缓存管理的关键 API。

5.2.2 扩展 PyTorch 缓存分配器。 KV缓存是张量的集合。在当前的深度学习框架（例如 PyTorch）中，通过 torch.empty 等 API 分配的张量带有预先分配的物理内存。这是因为 PyTorch 缓存分配器依赖于 cudaMalloc 接口（图 6）。依靠 CUDA 的低级 API 支持，我们扩展了 PyTorch 缓存分配器，以允许应用程序为张量保留虚拟内存缓冲区，而无需提前提交物理内存。我们将通过这些 API 分配的张量称为虚拟张量。

5.2.3 请求级KV缓存索引。虚拟张量表示最大批量大小 B 的层的 K 缓存（或 V 缓存）。在这些张量中，不同的请求占用不同的非重叠子区域（例如子张量）。我们使用唯一整数标识符 reqId 来定位请求的子张量，该标识符在 0 到 $B-1$ 的范围内（请注意，最多同时运行 B 请求）。请求的子张量在整个批次的虚拟张量中的 K 缓存（或 V 缓存）偏移量为 reqId $\times S$，其中 S 是工作器上请求的每层 K 缓存（或 V 缓存）的最大大小。请求标识reqId由vAttention分配。

5.3 使用 vAttention 为大语言模型服务

我们将 vAttention 构建为一个 Python 库，在内部使用 CUDA/C++ 扩展来与 CUDA 驱动程序交互。我们的库向服务框架公开了一组简单的 API（如图 6、表 4 和算法 1 所示）。为简单起见，我们从单个工作人员的角度讨论这些 API 的使用；所有工人的行为都一样。

5.3.1 初始设置。当服务框架启动时，每个模型工作线程加载 vAttention 库，并通过 init API（算法 1 中的第 4 行）使用模型参数 N, H, D, P、B 和首选页面组大小（第 6.2 节）对其进行配置。

Algorithm 1 Using vAttention in a serving framework.

```
1:  max_batch_size ← B
2:  cache_seq_len ← [0]*B
3:  req_batch_idx ← dict()
4:  vattention.init(config_params)
5:  while !request_pool.is_empty() do
6:      for R_i in new_requests do
7:          if can_schedule(R_i) then
8:              idx ← vattention.alloc_reqid()
9:              req_batch_idx[R_i] ← idx
10:             cache_seq_len[idx] ← prompt_len(R_i)
11:         end if
12:     end for
13:     vattention.step(cache_seq_len)
14:     model.forward()
15:     for R_i in active_requests do
16:         idx ← req_batch_idx[R_i]
17:         if is_complete(R_i) then
18:             cache_seq_len[idx] ← 0
19:             vattention.free_reqid(idx)
20:         else
21:             cache_seq_len[idx] + = 1
22:         end if
23:     end for
24: end while
```

在内部，vAttention在worker上保留了2个× N虚拟张量，如图5(a)所示，其中N是worker托管的层数。这些虚拟张量在服务应用程序的生命周期内保留。此外，vAttention还在初始化期间为每个worker预先分配物理内存页面。然而，此时这些页面还没有映射到KV缓存中。

5.3.2 安排新请求。当第一次安排新请求时，服务框架通过 alloc_reqid 从 vAttention 获取新的 reqId（第 8 行）。该请求的所有后续内存管理操作都使用此 reqId 进行标记。

5.3.3 模型执行。在调度一批执行之前，框架需要确保每个活动请求的 KV 缓存子张量都有物理内存支持（图 5（b）和（c））。为此，在将迭代的第一个内核分发到 GPU 之前，框架调用步骤 API（第 13 行），指定每个请求的当前上下文长度（对于每个不活动的 reqId，上下文长度设置为 0）。在内部，vAttention 确保在将执行返回到框架之前为每个活动 reqId 映射足够的物理页面。如果 vAttention 无法满足内存需求，它会返回失败，作为响应，服务框架可以抢占一个或多个请求以允许前进

(this is similar to vLLM's default behavior). We leave more sophisticated policies such as swapping out KV cache to CPU memory as future work.

Depending on whether a request is in the prefill phase or decode phase, different amount of physical memory may need to be mapped for a given iteration. The prefill phase processes the input tokens of a given prompt in parallel. Therefore, the amount of physical memory needed to be mapped depends on the number of prompt tokens being scheduled. If the total K cache size of all prompt tokens at one layer of the model is s and page-group size is t, then each worker needs to ensure that at least $(s+t-1)/t$ page-groups are mapped in each of the $2 \times N$ KV cache sub-tensors of the given `reqId`.

For a request in the decode phase, the number of new page-groups required is at most one per virtual tensor. This is because each iteration produces only one output token for a request. vAttention internally tracks the number of page-groups mapped for each request and maps new page-groups only when prior page-groups are about to be exhausted.

5.3.4 Request completion. A request terminates when it reaches user specified or the maximum context length supported by the model, or when the model produces a special end-of-sequence token. The framework notifies vAttention of a request's completion with `free_reqid` (line 19). Internally, vAttention may unmap the physical pages of a completed request or defer them to be freed later (§6.1.2).

Supporting continuous batching: Continuous batching poses one challenge in computing attention in our design. When a request from somewhere in the middle of a batch exits, it creates an unused hole in the virtual tensors of KV cache. This layout is not supported by implementations that expect the query (Q) and KV cache to be of the same size in batch (B) dimension. However, FlashAttention provides rich API support to address this issue; its argument `cache_batch_idx` allows Q and KV cache to have different batch sizes and also be arranged in arbitrary order (i.e., batch index 0 in Q can be mapped to batch index 1 in KV cache). vAttention benefits from this API support in terms of both performance and ease of programming; when the request composition of a batch changes, we update `cache_batch_idx` of running requests such that their Q tensors map to their respective KV cache based on their `reqId`.

6 Optimizations

There are two challenges in using CUDA virtual memory support for serving LLMs. First, invoking CUDA VMM APIs at runtime incurs high latency. Second, `cuMemCreate` currently allocates memory only at the granularity of large pages, i.e., multiples of 2MB. Use of large pages can waste physical memory due to internal fragmentation. This section details a set of simple-yet-effective optimizations that we introduce to overcome these challenges.

6.1 Hiding Latency of Memory Allocation

The serving framework invokes the `step` API in every iteration. The latency of `step` depends on the number of page-groups that need to be mapped into KV cache. Consider, for example, that the KV cache of one request needs to be extended for Yi-34B which has 60 layers. This requires 120 calls to `cuMemMap + cuMemSetAccess` each of which takes about 40 microseconds. Therefore, growing the KV cache of one request by new page-groups (two per layer) adds about 5 millisecond latency to the corresponding iteration. The latency overhead grows proportional to the number of requests that need new page-groups in a given iteration. We propose the following optimizations to hide this latency:

6.1.1 Overlapping memory allocation with compute (decode phase). We leverage the predictability of memory demand to overlap memory allocation with computation. In particular, note that each iteration produces a single output token for every decode request. Therefore, memory demand for a decode iteration is known ahead-of-time. Further, in the decode phase, a request requires at most one new page-group for each of its virtual tensors. vAttention keeps track of the current context length and how much physical memory is already mapped for each request. Using this information, it determines when a request would need more memory and uses a background thread to allocate new page-groups when the preceding iteration is executing. For example, consider that a request `R1` would require more physical memory in iteration `i`. When the serving framework invokes `step` API in iteration `i-1`, vAttention launches a background thread that maps page-groups for iteration `i`. Since per-iteration latency is typically in the range of 10s-100s of milliseconds, the background thread has enough time to prepare physical memory mappings for an iteration before it starts executing. This way, vAttention hides the latency of CUDA APIs by performing memory allocations out of the critical path.

6.1.2 Deferred reclamation + eager allocation (prefill phase). We observe that allocating physical memory for the prefill phase can be avoided in many cases. Consider that a request `R1` completed in iteration `i` and a new request `R2` joins the running batch in iteration `i+1`. To avoid allocating page-groups to `R2` from scratch, vAttention simply defers the reclamation of `R1`'s page-groups (Figure 5(d)) and assigns `R1`'s `reqId` to `R2`. This way, `R2` uses the same tensors for its KV cache that `R1` was using – which are already backed by physical pages (Figure 5(e)). Therefore, new allocations are required only if `R2`'s context length is higher than `R1`.

We further optimize the prefill phase by proactively allocating a small number of page-groups ahead of time. For this purpose, we try to keep a certain number of page-groups mapped into the virtual tensors of one of the inactive `reqId`. When a new request arrives, we allocate this `reqId`. At the same time, we identify a new `reqId` to be allocated next and

(这类似于 vLLM 的默认行为)。我们将更复杂的策略（例如将 KV 缓存交换到 CPU 内存）作为未来的工作。

根据请求是处于预填充阶段还是解码阶段，可能需要为给定迭代映射不同数量的物理内存。预填充阶段并行处理给定提示的输入标记。因此，需要映射的物理内存量取决于正在调度的提示令牌的数量。如果模型一层所有提示标记的总 K 缓存大小为 s，页面组大小为 t，则每个工作线程需要确保至少 $(s+t-1)/t$ 个页面组映射到给定 reqId 的 2 个 $\times N$KV 缓存子张量中。

对于解码阶段的请求，所需的新页组数量最多为每个虚拟张量一个。这是因为每次迭代仅为请求生成一个输出令牌。 vAttention 在内部跟踪为每个请求映射的页组数量，并仅在先前的页组即将耗尽时映射新的页组。

5.3.4 请求完成。当请求达到用户指定的或模型支持的最大上下文长度时，或者当模型生成特殊的序列结束标记时，请求终止。该框架通过 free_reqid 通知 vAttention 请求已完成（第 19 行）。在内部，vAttention 可以取消映射已完成请求的物理页面或推迟它们稍后释放（第 6.1.2 节）。

支持连续批处理：连续批处理对我们设计中的计算注意力提出了挑战。当来自批次中间某个位置的请求退出时，它会在 KV 缓存的虚拟张量中创建一个未使用的漏洞。期望查询 (Q) 和 KV 缓存在批量 (B) 维度中具有相同大小的实现不支持此布局。然而，FlashAttention提供了丰富的API支持来解决这个问题；它的参数cache_batch_idx允许Q和KV缓存具有不同的批大小，并且可以按任意顺序排列（即Q中的批索引0可以映射到KV缓存中的批索引1）。 vAttention 在性能和编程简易性方面受益于这种 API 支持；当批次的请求组成发生变化时，我们更新正在运行的请求的cache_batch_idx，以便它们的Q张量根据它们的reqId映射到它们各自的KV缓存。

6 优化

使用 CUDA 虚拟内存支持为 LLM 提供服务存在两个挑战。首先，在运行时调用 CUDA VMM API 会导致高延迟。其次，cuMemCreate 目前仅以大页面的粒度（即 2 MB 的倍数）分配内存。使用大页面可能会因内部碎片而浪费物理内存。本节详细介绍了我们为克服这些挑战而引入的一组简单但有效的优化。

6.1 隐藏内存分配的延迟

服务框架在每次迭代中调用步骤 API。步骤的延迟取决于需要映射到KV缓存的页组的数量。例如，考虑到一个请求的 KV 缓存需要扩展为 60 层的 Yi-34B。这需要对 cuMemMap + cuMemSetAccess 进行 120 次调用，每次调用大约需要 40 微秒。因此，通过新的页面组（每层两个）增加一个请求的 KV 缓存会给相应的迭代增加大约 5 毫秒的延迟。延迟开销与给定迭代中需要新页面组的请求数量成正比。我们建议进行以下优化来隐藏此延迟：

6.1.1 内存分配与计算重叠（解码阶段）。我们利用内存需求的可预测性将内存分配与计算重叠。特别要注意的是，每次迭代都会为每个解码请求生成一个输出令牌。因此，解码迭代的内存需求是提前已知的。此外，在解码阶段，请求最多需要为其每个虚拟张量一个新的页面组。 vAttention 跟踪当前上下文长度以及每个请求已映射多少物理内存。使用此信息，它可以确定请求何时需要更多内存，并在执行前面的迭代时使用后台线程分配新的页面组。例如，考虑请求 R1 在迭代 i 中需要更多物理内存。当服务框架在迭代 i-1 中调用步骤 API 时，vAttention 启动一个后台线程来映射迭代 i 的页面组。由于每次迭代延迟通常在 10 到 100 毫秒范围内，因此后台线程在开始执行之前有足够的时间为迭代准备物理内存映射。这样，vAttention 通过在关键路径之外执行内存分配来隐藏 CUDA API 的延迟。

6.1.2 延迟回收+急切分配（预填充阶段）。我们观察到，在许多情况下可以避免为预填充阶段分配物理内存。假设请求 R1 在迭代 i 中完成，新请求 R2 在迭代 i+1 中加入正在运行的批处理。为了避免从头开始将页面组分配给 R2，vAttention 只是推迟 R1 页面组的回收（图 5(d)）并将 R1 的 reqId 分配给 R2。这样，R2 对其 KV 缓存使用与 R1 使用的相同的张量——这些张量已经由物理页支持（图 5(e)）。因此，仅当 R2 的上下文长度高于 R1 时才需要新的分配。

我们通过提前主动分配少量页组来进一步优化预填充阶段。为此，我们尝试将一定数量的页面组映射到其中一个非活动 reqId 的虚拟张量中。当新的请求到达时，我们分配这个reqId。同时，我们确定接下来要分配的新 reqId，并且

eagerly map physical page-groups for it. In most cases, these eager allocations obviate the need to allocate physical memory in the critical path of prefill execution. Finally, we trigger memory reclamation only when the number of page-groups cached in vAttention falls below a certain threshold (e.g., less than 10% of GPU memory). We delegate both deferred reclamation and eager allocation to the background thread that the `step` API spawns.

6.2 Mitigating Internal Fragmentation

We mitigate internal fragmentation by reducing the granularity of physical memory allocation. NVIDIA GPUs natively support at least three page sizes: 4KB, 64KB and 2MB [22, 58, 65, 79]. Therefore, in principal, physical memory can be allocated in any multiple of 4KB sizes. The simplest way to achieve this would be to extend the existing CUDA VMM APIs (listed in Table 3) to also support allocating smaller pages (similar to how `mmap` in Linux supports multiple page sizes [52, 62]). Unfortunately, the CUDA VMM APIs are implemented in the closed-source NVIDIA drivers which makes it impossible for us to modify their implementation.

Fortunately, some part of NVIDIA drivers (particularly related to unified memory management) is open-source. Therefore, we implement a new set of APIs in the open-source NVIDIA drivers to mimic the same functionality that existing CUDA APIs provide but with support for multiple page sizes. The second column in Table 3 shows our new APIs: most of our APIs have a one-to-one relationship with existing CUDA APIs except for `vMemMap` that combines the functionality of `cuMemMap` and `cuMemSetAccess`, and `vMemRelease` that combines the functionality of `cuMemUnmap` and `cuMemRelease` for simplicity. In contrast to CUDA VMM APIs, our APIs allocate physical memory in 64KB, 128KB and 256KB sized page-groups. A serving framework can configure a desired page-group size in vAttention while initializing it (we use the standard 2MB pages if the configured page-group size is 2MB). Table 3 shows the latency of each API with different page-group sizes.

7 Evaluation

Our evaluation answers the following questions:

- How does vAttention impact the performance of attention kernels and the overall performance of prefill and decode phases (§7.1, §7.2)?
- How does vAttention impact the end-to-end LLM serving throughput (§7.3, §7.4, §7.5)?
- What is the effect of each of our optimizations (§7.6)?

Models and hardware: We evaluate three models Yi-6B [33], Llama-3-8B [21] and Yi-34B [32]. We conduct most of our evaluation on a single NVIDIA A100 GPU for Yi-6B, and two NVLink-connected A100 GPUs for Llama-3-8B and Yi-34B (see Table 5). Each GPU has 80GB physical memory. We

Model	Hardware	# Q Heads	# KV Heads	# Layers
Yi-6B	1 A100	32	4	32
Llama-3-8B	2 A100s	32	8	32
Yi-34B	2 A100s	56	8	60

Table 5. Models and hardware used for evaluation.

use tensor-parallelism degree of two (TP-2) for both Llama-3-8B and Yi-34B. To demonstrate the portability benefit of vAttention, we use 1–2 H100 GPUs.

Evaluation methodology: The computation and memory allocation pattern of the prefill and decode phases is substantially different [35, 46, 82]. Attention kernels used for these two phases are also different and hence we evaluate them separately. The prefill phase requires one time memory allocation potentially spanning multiple pages. In comparison, the decode phase requires incremental memory allocation over the lifetime of a request [51]. We define prefill throughput as the number of prompt tokens processed per second, and decode throughput as the number of tokens generated per second.

Serving framework: For a fair comparison, we use vLLM v0.2.7 as a common serving framework in all our experiments. We integrated state-of-the-art kernel libraries of both FlashAttention-2 v2.5.9 [1, 42, 43] and FlashInfer v0.4.0 [3] as attention back-ends into vLLM, and further added support for dynamic memory allocation via vAttention to their non-paged kernels. While FlashAttention-2 and FlashInfer are both based on the same underlying techniques (e.g., FlashDecoding [2]), they use different Block-Table formats; the former use a simple lookup table whereas the latter uses a compressed Block-Table to optimize lookups.

Baselines: We compare performance obtained by using the non-paged attention kernels (backed by vAttention for dynamic memory allocation), and their paged counterparts. In addition, we also compare against vLLM's decode kernel (note that vLLM does not have a paged prefill kernel). We also profiled each system to find its best performing configuration. Accordingly, we set KV cache block size to 16 for both vLLM and FlashInfer, and 256 for FlashAttention-2. Using a higher block size for vLLM increases its kernel latency by up to 1.9× as shown in Figure 3, and using a smaller block size for FlashAttention-2 paged kernel increases its latency by up to 9%. We find that the choice of page size does not affect vAttention as long as fragmentation is not a concern.

7.1 Prefill Evaluation

We evaluate 4 configurations for prefill: FA2_Paged, FI_Paged, FA2_vAttention and FI_vAttention. Configurations with the "_Paged" suffix represent the PagedAttention-based kernel and those with "_vAttention" use the non-paged kernels of the respective library. Figure 7 shows the prefill throughput for our models. We summarize key findings below.

急切地为其映射物理页组。在大多数情况下，这些急切分配消除了在预填充执行的关键路径中分配物理内存的需要。最后，仅当 vAttention 中缓存的页面组数量低于某个阈值（例如，小于 GPU 内存的 10%）时，我们才会触发内存回收。我们将延迟回收和急切分配委托给步骤 API 生成的后台线程。

6.2 减少内部碎片

我们通过降低物理内存分配的粒度来减少内部碎片。 NVIDIA GPU 本身支持至少三种页面大小：4KB、64KB 和 2MB [22、58、65、79]。因此，原则上，物理内存可以按 4KB 大小的任意倍数进行分配。实现这一目标的最简单方法是扩展现有的 CUDA VMM API（表 3 中列出）， 以支持分配较小的页面（类似于 Linux 中的 mmap 支持多种页面大小 [52, 62]）。不幸的是，CUDA VMM API 是在闭源 NVIDIA 驱动程序中实现的，这使得我们无法修改它们的实现。

幸运的是，NVIDIA 驱动程序的某些部分（特别是与统一内存管理相关）是开源的。因此，我们在开源 NVIDIA 驱动程序中实现了一组新的 API，以模仿现有 CUDA API 提供的相同功能，但支持多种页面大小。表 3 中的第二列显示了我们的新 API：我们的大多数 API 与现有 CUDA API 都具有一对一的关系，但 vMemMap 结合了 cuMemMap 和 cuMemSetAccess 的功能，而 vMemRelease 结合了 cuMemUnmap 和 cuMemRelease 的功能（为简单起见）。与 CUDA VMM API 相比，我们的 API 以 64KB、128KB 和 256KB 大小的页组分配物理内存。服务框架可以在初始化 vAttention 时在 vAttention 中配置所需的页面组大小（如果配置的页面组大小为 2MB，我们使用标准 2MB 页面）。表 3 显示了具有不同页组大小的每个 API 的延迟。

7 评价

我们的评估回答了以下问题：

- vAttention 如何影响注意力内核的性能以及预填充和解码阶段的整体性能（第 7.1 节，第 7.2 节）？
- vAttention 如何影响端到端 LLM 服务吞吐量（§7.3、§7.4、§7.5）？
- 我们每项优化的效果是什么（第 7.6 节）？

模型和硬件：我们评估了三种模型 Yi-6B [33]、Llama-3-8B [21] 和 Yi-34B [32]。我们对 Yi-6B 的单个 NVIDIA A100 GPU 以及 Llama-3-8B 和 Yi-34B 的两个 NVLink 连接的 A100 GPU 进行大部分评估（参见表 5）。每个GPU有80GB物理内存。我们

Model	Hardware	# Q Heads	# KV Heads	# Layers
Yi-6B	1 A100	32	4	32
Llama-3-8B	2 A100s	32	8	32
Yi-34B	2 A100s	56	8	60

表 5. 用于评估的模型和硬件。

对 Llama-3-8B 和 Yi-34B 均使用二度张量并行度 (TP-2)。为了演示 vAttention 的可移植性优势，我们使用 1-2 个 H100 GPU。

评估方法：预填充和解码阶段的计算和内存分配模式有很大不同[35,46,82]。这两个阶段使用的注意力内核也不同，因此我们分别评估它们。预填充阶段需要一次内存分配，可能跨越多个页面。相比之下，解码阶段需要在请求的生命周期内增量内存分配[51]。我们将预填充吞吐量定义为每秒处理的提示令牌数，并将解码吞吐量定义为每秒生成的令牌数。

服务框架：为了公平比较，我们在所有实验中使用 vLLM v0.2.7 作为通用服务框架。我们将最先进的 FlashAttention-2 v2.5.9 [1, 42, 43] 和 FlashInfer v0.4.0 [3] 的内核库作为注意力后端集成到 vLLM 中，并进一步通过 vAttention 添加对动态内存分配的支持到其非分页内核。虽然 FlashAttention-2 和 FlashInfer 都基于相同的底层技术（例如 FlashDecoding [2]）， 但它们使用不同的块表格式；前者使用简单的查找表，而后者使用压缩的块表来优化查找。

基线：我们比较使用非分页注意力内核（由 vAttention 支持动态内存分配）及其分页对应内核获得的性能。此外，我们还与 vLLM 的解码内核进行比较（请注意，vLLM 没有分页预填充内核）。我们还对每个系统进行了分析，以找到其最佳性能配置。因此，我们将 vLLM 和 FlashInfer 的 KV 缓存块大小设置为 16，将 FlashAttention-2 的 KV 缓存块大小设置为 256。对 vLLM 使用较大的块大小会使其内核延迟最多增加 1.9×，如图 3 所示，而对 FlashAttention-2 分页内核使用较小的块大小会使其延迟最多增加 9%。我们发现，只要不考虑碎片，页面大小的选择就不会影响 vAttention。

7.1 预填充评估

我们评估了 4 种预填充配置：FA2_Paged、FI_Paged、FA2_vAttention 和 FI_vAttention。带有“_Paged”后缀的配置表示基于 PagedAttention 的内核，而带有“_vAttention”后缀的配置则使用相应库的非分页内核。图 7 显示了我们模型的预填充吞吐量。我们总结了以下主要发现。

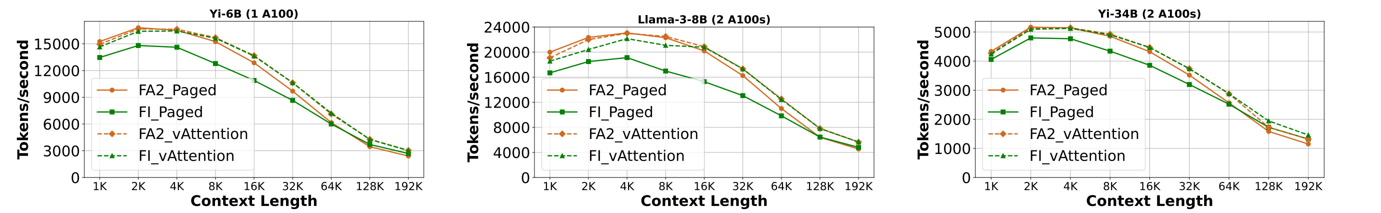


Figure 7. Prefill throughput. vAttention backed systems outperform the paged counterparts of both FlashAttention-2 and FlashInfer. Throughput for longer contexts is lower due to the quadratic complexity of prefill attention.

Model	Context Length	FlashAttention-2 Paged	FlashAttention-2 vAttention	FlashInfer Paged	FlashInfer vAttention
Yi-6B	64K	10.6 (7.0)	9.1 (5.5)	10.9 (6.0)	9.1 (5.4)
	128K	37.9 (30.3)	30.5 (23.1)	35.4 (25.4)	30.7 (23.3)
	192K	81.5 (70.0)	64.6 (53.6)	73.0 (58.3)	65.1 (53.6)
Llama-3-8B	64K	6.0 (3.4)	5.2 (2.7)	6.7 (3.0)	5.3 (2.8)
	128K	20.4 (15.4)	16.8 (11.6)	20.3 (12.8)	16.8 (11.4)
	192K	43.3 (35.6)	34.8 (26.9)	40.9 (29.7)	34.7 (26.7)
Yi-34B	64K	25.5 (13.2)	22.8 (10.3)	26.0 (11.2)	22.7 (10.1)
	128K	82.8 (56.9)	68.4 (43.2)	76.0 (46.7)	67.4 (42.5)
	192K	170.7 (131.8)	136.9 (98.8)	148.8 (104.7)	134.6 (96.5)

Table 6. Prefill completion and attention (in parenthesis) time with different attention back-ends (unit: seconds).

Model	BS	vLLM	FA2_Paged	FI_Paged	FA2_vAttention
Yi-6B	16	32.3	11.5	15.2	11.3
	32	64.1	25.5	25.4	25.3
Llama-3-8B	16	17.8	11.9	12.1	11.8
	32	35.3	25.4	23.23	25.3
Yi-34B	12	41.4	17.4	24.1	17.4
	16	55.1	21.7	28.8	21.8

Table 7. Total latency of attention kernel (sum of all layers) per decode iteration (in milliseconds, BS = batch size).

Small contexts: For small contexts, prefill cost is dominated by the linear operators, i.e., attention's contribution is relatively low [36]. Hence, even though vAttention helps speed up attention computation, the throughput of both paged and vAttention back-ends is nearly identical in FlashAttention-2. However, for FlashInfer, we find that using vAttention helps improve prefill throughput even for small contexts. This is because FlashInfer incurs various other sources of overhead in the paged version. First, appending a new K or V tensor to the KV cache requires a single tensor copy operation in vAttention, whereas in a paged implementation, it requires appending one block at a time (the copy operation has been optimized for FlashAttention-2 by vLLM [13]). Second, FlashInfer involves creation and deletion of a few objects for its compressed Block-Tables in every iteration. vAttention avoids such overheads because it maintains KV cache's virtual contiguity, eliminating the need for a Block-Table.

Long contexts: The contribution of attention computation becomes significant at 16K and higher context lengths in our experiments. Therefore, even for FlashAttention-2 back-end, vAttention outperforms the paged counterpart. For example, at context length 192K, FA2_vAttention outperforms FA2_Paged by 1.24×, 1.26× and 1.24× for Yi-6B, Llama-3-8B and Yi-34B, respectively. Similarly, for FlashInfer back-ends, FI_vAttention improves prefill throughput by up to 1.25× and 1.36× for Yi-6B and Llama-3-8B (context length 16K), and 1.17× for Yi-34B (context length 32K).

Attention time: vAttention's improvement in prefill throughput is primarily due to faster attention kernels enabled by a (virtually) contiguous KV cache. This is because the prefill phase of a long prompt is primarily dominated by attention computation, as can be observed by comparing the numbers inside parenthesis with total prefill completion time in Table 6. For FlashAttention-2, nearly all the gains of vAttention are due to faster attention kernels, e.g., prefill gains of 1.5 seconds, 7.4 seconds and 16.9 seconds for Yi-6B are all due to gains in attention computation. vAttention enabled kernels also help with the FlashInfer back-end. In addition, FI_Paged also has other sources of overheads, e.g., in the 14 seconds of total savings (Yi-34B, 192K context), only 7 seconds is due to attention and the rest is due to other sources.

7.2 Decode Evaluation

For decodes, in addition to FA2_Paged and FI_Paged, we also evaluate the throughput obtained with vLLM's decode kernel (the first ever kernel to support PagedAttention). For vAttention, we use FlashAttention-2's non-paged kernel. Unfortunately FlashInfer's non-paged decode kernel has significantly higher latency (up to 14.6×) compared to all these other kernels. Hence, while vAttention can support dynamic memory allocation for FlashInfer's non-paged decode kernel, we omit it for evaluation in this section.

Figure 8 shows the decode throughput of Yi-6B, Llama-3-8B and Yi-34B with varying batch size up to 32 (except for Yi-34B which runs out of memory for batch size 32). We set the initial context length of each request to 16K tokens and calculate decode throughput based on the mean latency of 400 decode iterations.

First, vAttention is on par with the best of PagedAttention as shown by FA2_Paged and FA2_vAttention in Figure 8. In comparison, FI_Paged has somewhat lower throughput and vLLM is the worst for all models and configurations.

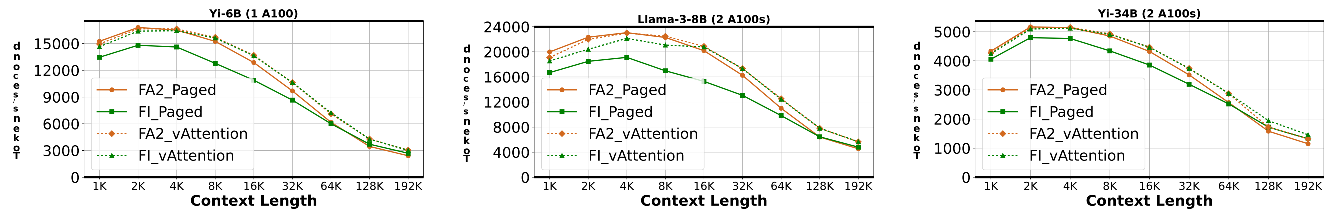


图 7. 预填充吞吐量。 vAttention 支持的系统优于 FlashAttention-2 和 FlashInfer 的分页系统。由于预填充注意力的二次复杂度，较长上下文的吞吐量较低。

Model	Context Length	FlashAttention-2 Paged	FlashAttention-2 vAttention	FlashInfer Paged	FlashInfer vAttention
Yi-6B	64K	10.6 (7.0)	9.1 (5.5)	10.9 (6.0)	9.1 (5.4)
	128K	37.9 (30.3)	30.5 (23.1)	35.4 (25.4)	30.7 (23.3)
	192K	81.5 (70.0)	64.6 (53.6)	73.0 (58.3)	65.1 (53.6)
Llama-3 -8B	64K	6.0 (3.4)	5.2 (2.7)	6.7 (3.0)	5.3 (2.8)
	128K	20.4 (15.4)	16.8 (11.6)	20.3 (12.8)	16.8 (11.4)
	192K	43.3 (35.6)	34.8 (26.9)	40.9 (29.7)	34.7 (26.7)
Yi-34B	64K	25.5 (13.2)	22.8 (10.3)	26.0 (11.2)	22.7 (10.1)
	128K	82.8 (56.9)	68.4 (43.2)	76.0 (46.7)	67.4 (42.5)
	192K	170.7 (131.8)	136.9 (98.8)	148.8 (104.7)	134.6 (96.5)

表 6. 不同注意力后端的预填充完成和注意力（括号内）时间（单位：秒）。

Model	BS	vLLM	FA2_Paged	FI_Paged	FA2_vAttention
Yi-6B	16	32.3	11.5	15.2	11.3
	32	64.1	25.5	25.4	25.3
Llama-3 -8B	16	17.8	11.9	12.1	11.8
	32	35.3	25.4	23.23	25.3
Yi-34B	12	41.4	17.4	24.1	17.4
	16	55.1	21.7	28.8	21.8

表 7. 每次解码迭代的注意力内核总延迟（所有层的总和）（以毫秒为单位，BS = 批量大小）。

小上下文：对于小上下文，预填充成本由线性算子主导，即注意力的贡献相对较低[36]。因此，尽管 vAttention 有助于加速注意力计算，但 FlashAttention-2 中分页后端和 vAttention 后端的吞吐量几乎相同。然而，对于 FlashInfer，我们发现使用 vAttention 有助于提高预填充吞吐量，即使对于小型上下文也是如此。这是因为 FlashInfer 在分页版本中会产生各种其他来源的开销。首先，将新的 K 或 V 张量附加到 KV 缓存需要在 vAttention 中进行单个张量复制操作，而在分页实现中，它需要一次附加一个块（复制操作已由 vLLM [13] 针对 FlashAttention-2 进行了优化）。其次，FlashInfer 涉及在每次迭代中为其压缩块表创建和删除一些对象。 vAttention 避免了此类开销，因为它维护了 KV 缓存的虚拟连续性，从而消除了对块表的需要。

长上下文：在我们的实验中，注意力计算的贡献在 16K 和更高上下文长度时变得显着。因此，即使对于 FlashAttention-2 后端，vAttention 的性能也优于分页版本。例如，在上下文长度为 192K 时，对于 Yi-6B、Llama-3-8B 和 Yi-34B，FA2_vAttention 的性能分别比 FA2_Paged 好 1.24×、1.26× 和 1.24×。同样，对于 FlashInfer 后端，FI_vAttention 将 Yi-6B 和 Llama-3-8B（上下文长度 16K）的预填充吞吐量提高了 1.25× 和 1.36×，将 Yi-34B（上下文长度 32K）的预填充吞吐量提高了 1.17×。

注意力时间：vAttention 在预填充吞吐量方面的改进主要是由于（虚拟）连续的 KV 缓存启用了更快的注意力内核。这是因为预填充长提示的阶段主要由注意力计算主导，通过将括号内的数字与表 6 中的总预填充完成时间进行比较可以观察到。对于 FlashAttention-2，几乎所有 vAttention 的增益都归因于更快的注意力内核，例如，Yi-6B 的 1.5 秒、7.4 秒和 16.9 秒的预填充增益都是由于注意力计算的增益。支持 vAttention 的内核还有助于 FlashInfer 后端。此外，FI_Paged还有其他开销来源，例如，在总共节省的14秒中（Yi-34B，192K上下文），只有7秒是由于注意力，其余是由于其他来源。

7.2 解码评估

对于解码，除了 FA2_Paged 和 FI_Paged 之外，我们还评估了使用 vLLM 解码内核（第一个支持 PagedAttention 的内核）获得的吞吐量。对于 vAttention，我们使用 FlashAttention-2 的非分页内核。不幸的是，与所有其他内核相比，FlashInfer 的非分页解码内核具有明显更高的延迟（高达 14.6×）。因此，虽然 vAttention 可以支持 FlashInfer 的非分页解码内核的动态内存分配，但我们在本节中省略了它的评估。

图 8 显示了 Yi-6B、Llama-3-8B 和 Yi-34B 的解码吞吐量，其批处理大小最多为 32（Yi-34B 除外，它在批处理大小为 32 时内存不足）。我们将每个请求的初始上下文长度设置为 16K 令牌，并根据 400 次解码迭代的平均延迟计算解码吞吐量。

首先，vAttention 与最好的 PagedAttention 相当，如图 8 中的 FA2_Paged 和 FA2_vAttention 所示。相比之下，FI_Paged 的吞吐量稍低，而 vLLM 对于所有模型和配置来说都是最差的。

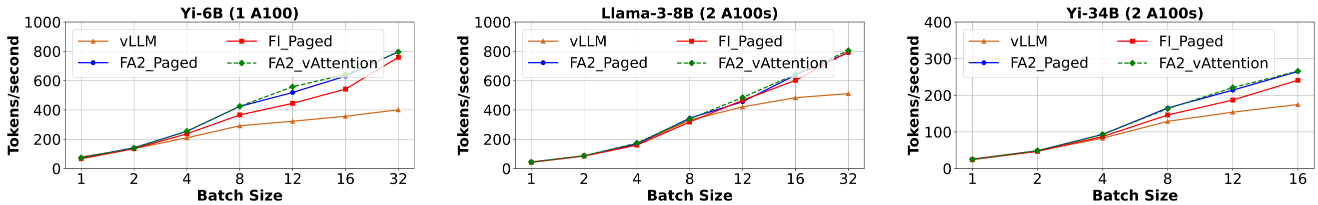


Figure 8. Decode throughput. FA2_vAttention is on par with FA2_Paged (note the overlapping lines) which is the best among all PagedAttention based alternatives, while outperforming FI_Paged and vLLM.

For example, FA2_Paged and FA2_vAttention outperform vLLM by up to 1.99×, 1.58× and 1.53× for Yi-6B, Llama-3-8B and Yi-34B, respectively. The primary reason is that vLLM's decode kernel has significantly higher latency than the FlashAttention-2 based kernels; while FlashAttention-2 has continuously adopted new optimizations (e.g., FlashDecoding [2]), vLLM has lagged behind. For example, Table 7 shows that vLLM's PagedAttention kernel incurs up to 2.8×, 1.5×, and 2.5× higher latency than FlashAttention-2 kernels for Yi-6B, Llama-3-8B and Yi-34B. This is despite vLLM being in an actively maintained open-source serving stack, as well as being used by various companies for serving LLMs. This is an important result that underlines the importance of low software complexity and portability.

Second, relative gains of FA2_vAttention and FA2_Paged increase over vLLM with the batch size, e.g., as batch size increases from 4 to 32 for Llama-3-8B, relative gains increase from 1.05× to 1.58×. This is because the latency of a decode attention kernel is proportional to the total number of tokens in the batch [34]. Therefore, the contribution of attention kernel in the overall latency – hence gains with a more efficient kernel – increase with the batch size. Further, for the same reasons as discussed in §7.1 (faster attention and lower CPU overhead), FA2_vAttention delivers up to 1.23× higher throughput than FI_Paged (Yi-6B, batch size 12).

Finally, note that vAttention is only as good as the state-of-the-art PagedAttention for decode throughput, as compared to prefills where it outperforms PagedAttention. This is because in case of decode attention, paged and non-paged kernels have similar latency as shown in Table 7. We believe this is due to memory bound nature of decode attention, i.e., memory stalls make it possible to hide the effect of additional compute that paging support requires. However, hiding compute overhead in a prefill attention kernel is hard because it is already compute bound (see Table 6 for vAttention's gains in the prefill kernel).

7.3 End-to-end Performance: Offline Scenarios

We measure the end-to-end system throughput in an offline scenario as the number of requests completed per minute. We evaluate a total of 427 long-context requests from the arXiv-Summarization workload trace wherein the total context length per-request varies from 64K tokens to 192K tokens

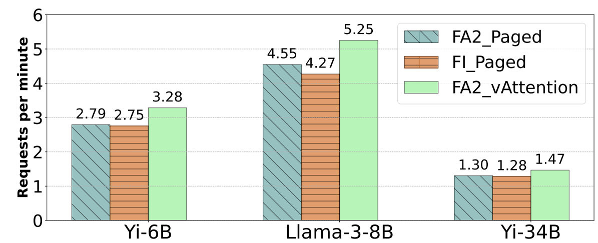


Figure 9. Offline inference throughput while serving long-context requests from the arXiv-Summarization dataset [11].

and the number of output tokens per-request varies from 17 to 5153. The mean prefill to decode token ratio is 356. Figure 9 shows throughput for different attention backends for all three models. FA2_vAttention outperforms FA2_Paged by 1.18×, 1.15× and 1.13×, and FI_Paged by 1.19×, 1.23×, and 1.14× for Yi-6B, Llama-3-8B and Yi-34B, respectively. In general, vAttention's performance gains are proportional to the context length and the prefill to decode token ratio (P:D ratio); these factors determine how much prefill attention contributes to the total runtime. A higher P:D ratio as well as longer context lengths indicate that the workload is more prefill bound. vAttention provides higher gains in such cases.

7.4 End-to-end Performance: Online Scenarios

We evaluate an online inference scenario serving long-context requests from arXiv-Summarization [11]. In total, we run 512 requests wherein the per-request input context length varies from 22K to 45K tokens (mean 29K), the number of decode tokens varies from 6 to 3250 (mean 348) and the mean P:D ratio is 129. We evaluate performance near the serving capacity of the system which denotes the maximum load a system can handle without incurring high queuing delays [35, 51]. We vary input load (queries-per-second or QPS) based on Poisson distribution and schedule requests in first-come-first-serve order. Figure 10 shows the CDF of the end-to-end request execution latency for this experiment. We see that vAttention consistently outperforms both baselines. For example, FA2_vAttention reduces the median request execution latency over FA2_Paged by up to 42% (QPS 0.25) for Yi-6B, by up to 28% (QPS 0.3) for Llama-3-8B and by up to 29% for Yi-34B (QPS 0.1). The primary reason for vAttention's

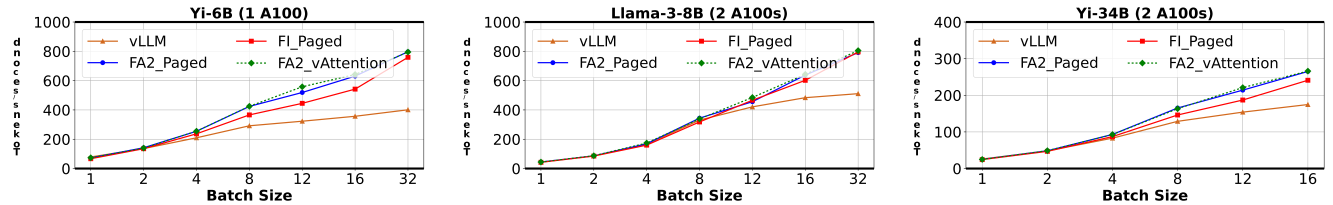


图 8. 解码吞吐量。 FA2_vAttention 与 FA2_Paged 相当（注意重叠线），这是所有基于 PagedAttention 的替代方案中最好的，同时优于 FI_Paged 和 vLLM。

例如，对于 Yi-6B、Llama-3-8B 和 Yi-34B，FA2_Paged 和 FA2_vAttention 的性能分别优于 vLLM 高达 1.99×、1.58× 和 1.53×。主要原因是 vLLM 的解码内核的延迟明显高于基于 FlashAttention-2 的内核；虽然 FlashAttention-2 不断采用新的优化（例如 FlashDecoding [2]），但 vLLM 却落后了。例如，表 7 显示，对于 Yi-6B、Llama-3-8B 和 Yi-34B，vLLM 的 PagedAttention 内核的延迟比 FlashAttention-2 内核高出 2.8×、1.5× 和 2.5×。尽管 vLLM 处于积极维护的开源服务堆栈中，并且被多家公司用于为 LLM 提供服务，但情况仍然如此。这是一个重要的结果，强调了低软件复杂性和可移植性的重要性。

其次，FA2_vAttention 和 FA2_Paged 相对于 vLLM 的相对增益随着批量大小的增加而增加，例如，当 Llama-3-8B 的批量大小从 4 增加到 32 时，相对增益从 1.05× 增加到 1.58×。这是因为解码注意力内核的延迟与批次中的令牌总数成正比[34]。因此，注意力内核对整体延迟的贡献（因此通过更高效的内核获得的收益）随着批量大小的增加而增加。此外，出于与第 7.1 节中讨论的相同原因（更快的注意力和更低的 CPU 开销），FA2_vAttention 的吞吐量比 FI_Paged（Yi-6B，批量大小 12）高出 1.23×。

最后，请注意，vAttention 在解码吞吐量方面仅与最先进的 PagedAttention 一样好，与预填充相比，它的性能优于 PagedAttention。这是因为在解码注意力的情况下，分页和非分页内核具有相似的延迟，如表 7 所示。我们认为这是由于解码注意力的内存限制性质，即内存停顿可以隐藏分页支持所需的额外计算的影响。然而，在预填充注意力内核中隐藏计算开销很困难，因为它已经受到计算限制（有关预填充内核中 vAttention 的增益，请参阅表 6）。

7.3 端到端性能：离线场景

我们以每分钟完成的请求数来衡量离线场景下的端到端系统吞吐量。我们评估了来自 arXiv-Summarization 工作负载跟踪的总共 427 个长上下文请求，其中每个请求的总上下文长度从 64K 令牌到 192K 令牌不等

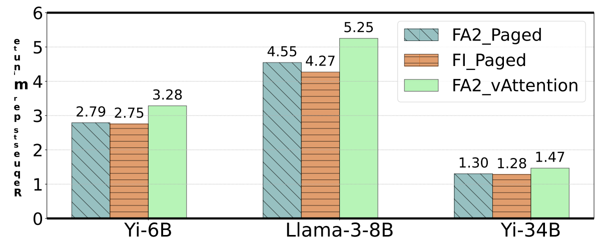


图 9. 服务来自 arXiv-Summarization 数据集 [11] 的长上下文请求时的离线推理吞吐量。

每个请求的输出令牌数量从 17 到 5153 不等。预填充与解码令牌的平均比率为 356。图 9 显示了所有三个模型的不同注意力后端的吞吐量。 FA2_vAttention 的性能优于 FA2_Paged 1.18×、1.15× 和 1.13×，FI_Paged 的性能优于 1.19×、1.23× 和 1.14×分别为 Yi-6B、Llama-3-8B 和 Yi-34B。一般来说，vAttention 的性能增益与上下文长度和预填充解码令牌比率（P:D 比率）成正比；这些因素决定了预填充注意力对总运行时间的贡献程度。较高的 P:D 比率以及较长的上下文长度表明工作负载更受预填充限制。在这种情况下，vAttention 可以提供更高的收益。

7.4 端到端性能：在线场景

我们评估了一个服务于 arXiv-Summarization [11] 长上下文请求的在线推理场景。总共，我们运行了 512 个请求，其中每个请求的输入上下文长度从 22K 到 45K 个令牌（平均 29K）不等，解码令牌的数量从 6 到 3250 个（平均 348）不等，平均 P:D 比率为 129。我们评估接近系统服务容量的性能，这表示系统在不产生高排队延迟的情况下可以处理的最大负载 [35, 51]。我们根据泊松分布改变输入负载（每秒查询数或 QPS），并按照先到先服务的顺序安排请求。图 10 显示了该实验的端到端请求执行延迟的 CDF。我们看到 vAttention 始终优于两个基线。例如，与 FA2_Paged 相比，FA2_vAttention 将 Yi-6B 的中位请求执行延迟降低了 42%（QPS 0.25），将 Llama-3-8B 降低了 28%（QPS 0.3），将 Yi-34B 降低了 29%（QPS 0.1）。 vAttention 的主要原因

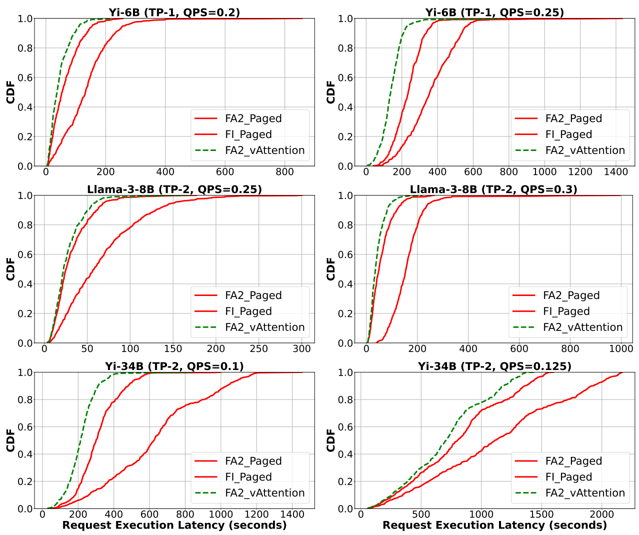


Figure 10. CDF of end-to-end request execution latency in online inference under varying load.

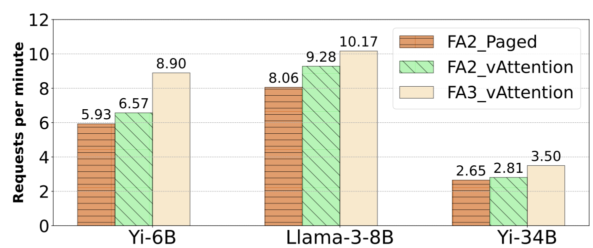


Figure 11. Offline inference throughput on H100 GPUs.

performance gain is that it can compute the prefill phase of new requests faster than the PagedAttention-based systems which substantially reduces queuing delays. Consistent with our prior results, FI_Paged is slower than FA2_Paged and hence our gains our FI_Paged are relatively higher compared to our gains over FA2_Paged.

7.5 Exemplifying Portability: FlashAttention-3

We demonstrate the portability benefit of vAttention with the recently released FA3 kernel [67]. FA3 is optimized for the NVIDIA Hopper architecture and did not support PagedAttention when released. Therefore, dynamic memory allocation via PagedAttention is not feasible for FA3 at the time of writing this paper (integration of FA3 into vLLM is also a work in progress [16, 26]). vAttention not only enables dynamic memory allocation with FA3, it also requires no code changes to deploy FA3. Figure 11 shows the offline inference throughput of our models with 1–2 H100 GPUs (Yi-6B deployed on a single GPU and the other models on two GPUs each) on the same arXiv-Summarization-based workload as evaluated in §7.3. FA3 with vAttention provides an additional

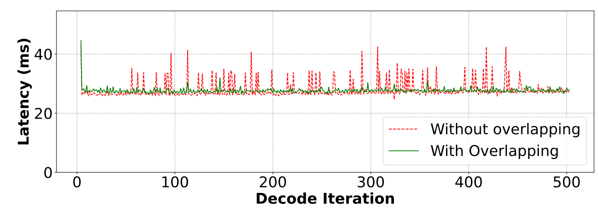


Figure 12. Latency of decode iterations with and without overlapping memory allocation with compute (batch size=32, context length=4K–8K, model: Llama-3-8B)

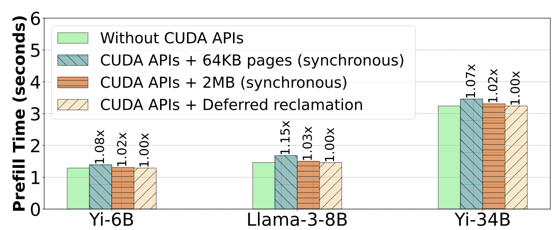


Figure 13. Prefill completion time of a single prompt of 16K tokens with different memory allocation strategies.

speedup of up to 1.35× (Yi-6B) over FA2_vAttention which is already up to 1.15× faster than FA2_Paged.

7.6 Ablation Studies

7.6.1 Hiding allocation latency. Figure 12 shows the latency of more than 500 decode iterations of Llama-3-8B. For this experiment, we used a batch of 32 requests with per request prefill context length varying between 2K to 3K tokens. It is evident that overlapping memory allocation with compute effectively hides the latency of CUDA VMM APIs. We used 2MB pages for this experiment to show that even the worst case memory allocation latency can be hidden by overlapping it with compute (Table 3 shows that 2MB pages incur highest latency). In contrast, allocating memory synchronously via CUDA APIs leads to frequent latency spikes of 5 to 15 milliseconds, depending on how many requests require physical memory allocation in a given iteration.

7.6.2 Deferred reclamation. Figure 13 shows that synchronous memory allocation incurs prefill overhead of up to 1.15× with 64KB pages and up to 1.03× with 2MB pages. In most cases, deferred reclamation eliminates the need to invoke CUDA VMM APIs for prefills because a newly arrived request can simply re-use physical memory allocated to a prior request. This way, deferred reclamation ensures that prefill latency is not affected by memory allocation.

7.6.3 Effect of page size. In many applications, use of smaller pages can potentially degrade performance due to TLB thrashing [52, 62, 64, 70]. We find that this is not the case for LLM inference. For example, Figure 14 shows that

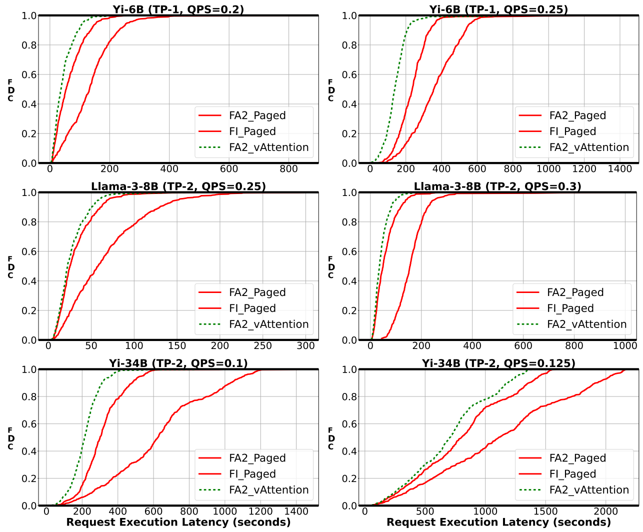


图 10. 变化负载下在线推理中端到端请求执行延迟的 CDF。

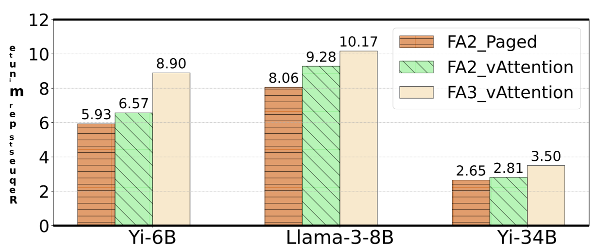


图 11. H100 GPU 上的离线推理吞吐量。

性能增益在于它可以比基于 PagedAttention 的系统更快地计算新请求的预填充阶段，从而大大减少排队延迟。与我们之前的结果一致，FI_Paged 比 FA2_Paged 慢，因此与 FA2_Paged 相比，我们的 FI_Paged 收益相对较高。

7.5 可移植性示例：FlashAttention-3

我们通过最近发布的 FA3 内核展示了 vAttention 的可移植性优势 [67]。 FA3 针对 NVIDIA Hopper 架构进行了优化，发布时并不支持 Page-dAttention。因此，在撰写本文时，通过 PagedAttention 进行动态内存分配对于 FA3 来说是不可行的（将 FA3 集成到 vLLM 也是一项正在进行的工作 [16, 26]）。 vAttention 不仅支持 FA3 的动态内存分配，而且无需更改代码即可部署 FA3。图 11 显示了我们的模型在相同的基于 arXiv-Summarization 的工作负载上的离线推理吞吐量，该模型具有 1-2 个 H100 GPU（Yi-6B 部署在单个 GPU 上，其他模型部署在两个 GPU 上），如第 7.3 节中评估的那样。带有 vAttention 的 FA3 提供了额外的

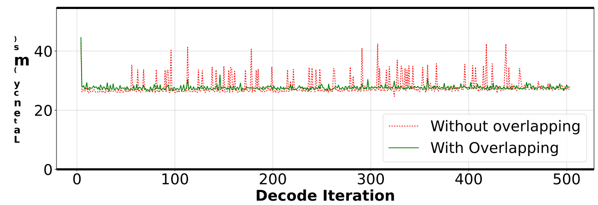


图 12. 有或没有与计算重叠内存分配的解码迭代延迟（批量大小=32，上下文长度=4K–8K，模型：Llama-3-8B）

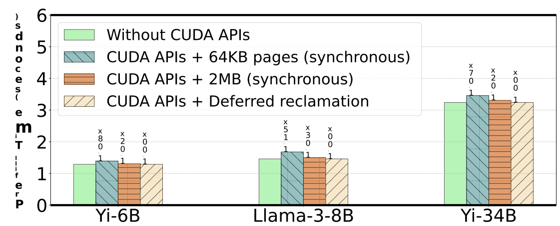


图 13. 使用不同内存分配策略的 16K 令牌的单个提示的预填充完成时间。

与 FA2_vAttention 相比，加速高达 1.35× (Yi-6B)，而 FA2_vAttention 已经比 FA2_Paged 快 1.15×。

7.6 消融研究

7.6.1 隐藏分配延迟。图 12 显示了 Llama-3-8B 超过 500 次解码迭代的延迟。在本实验中，我们使用了一批 32 个请求，每个请求预填充上下文长度在 2K 到 3K 令牌之间变化。很明显，内存分配与计算的重叠有效地隐藏了 CUDA VMM API 的延迟。我们在本实验中使用 2MB 页面，以表明即使是最坏情况的内存分配延迟也可以通过与计算重叠来隐藏（表 3 显示 2MB 页面会产生最高延迟）。相比之下，通过 CUDA API 同步分配内存会导致频繁的 5 到 15 毫秒的延迟峰值，具体取决于给定迭代中有多少请求需要物理内存分配。

7.6.2 延期回收。图 13 显示，同步内存分配会产生高达 1.15×（对于 64KB 页面）和高达 1.03×（对于 2MB 页面）的预填充开销。在大多数情况下，延迟回收无需调用 CUDA VMM API 进行预填充，因为新到达的请求可以简单地重复使用分配给先前请求的物理内存。这样，延迟回收可确保预填充延迟不受内存分配的影响。

7.6.3 页面大小的影响。在许多应用程序中，使用较小的页面可能会由于 TLB 抖动而降低性能 [52,62,64,70]。我们发现 LLM 推理并非如此。例如，图 14 显示

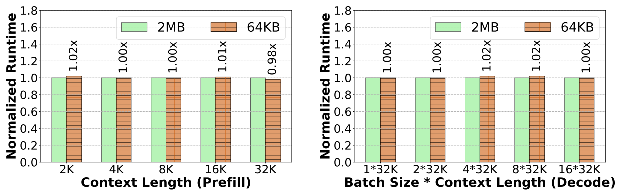


Figure 14. Effect of page size on the runtime of FlashAttention-2's prefill (left) and decode (right) attention kernels (model: Llama-3-8B).

Model	**Block Size (# Tokens in a page-group)**			
	64KB	128KB	256KB	2MB
Yi-6B (TP-1)	64	128	256	2048
Yi-6B (TP-2)	128	256	512	4096
Llama-3-8B (TP-1)	32	64	128	1024
Llama-3-8B (TP-2)	64	128	256	2048
Yi-34B (TP-1)	32	64	128	1024
Yi-34B (TP-2)	64	128	256	2048

Table 8. KV cache block size as a function of page-group size and degree of tensor parallelism.

the execution latency of an attention kernel remains largely unaffected when the KV cache is allocated using 64KB pages, compared to using 2MB pages. In a separate experiment, we find that these results are also consistent with very large models, e.g., Llama-3-70B and GPT-3-175B. We attribute this to the regular computation pattern of the attention operator as well as the hand-tuned implementations that explicitly try to avoid irregular memory accesses.

Table 8 shows the block size i.e., number of tokens per physical page-group. Smaller page-groups enable fine-grained allocation, approaching vLLM's recommended block size of 16–32 (the minimum block size in FlashAttention-2 is 256 – higher than vAttention). This shows that vAttention is as effective in reducing fragmentation as PagedAttention. In terms of end-to-end system performance, we find that 2MB pages are good enough for many online serving scenarios where latency constraints prevent use of very high batch sizes. In contrast, smaller page-groups are better for throughput-oriented scenarios where maximizing batch size is important to obtain peak performance. For example, using 2MB pages could serve batch sizes of up to 187, 203 and 56 for Yi-6B, Llama-3-8B and Yi-34B on a dynamic trace (dataset: OpenChat [74], load: 7 queries per seconds). In contrast, 64KB pages helped serve batch sizes of up to 240, 258 and 68 for our models as shown in Figure 15.

7.6.4 Memory allocation bandwidth. Table 9 shows that even with 64KB pages (our smallest), vAttention can allocate as much as 7.6GB physical memory per second per GPU. This is more than an order of magnitude higher than the maximum memory allocation rate of 750MB per second of

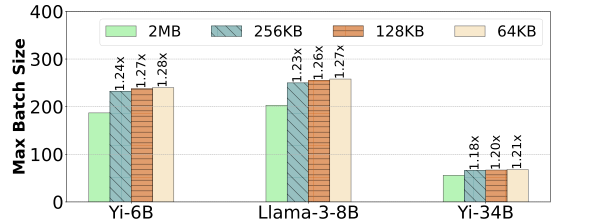


Figure 15. Maximum batch size obtained with different page-group sizes for a dynamic workload.

Config.	64KB	128KB	256KB	2MB
TP-1	7.59	14.56	27.04	35.17
TP-2	15.18	29.12	54.08	70.34

Table 9. Physical memory allocation bandwidth (GB per second) with varying allocation granularity.

decodes (Figure 4). Larger page-group sizes and higher TP dimensions increase the memory allocation rate proportionally. Therefore, memory allocation bandwidth of CUDA VMM APIs is more than sufficient for LLM inference.

8 Discussion

8.1 Managing KV cache via Unified Memory

To enable dynamic memory allocation, we also considered leveraging unified memory via `cudaMallocManaged` [58, 60]. However, we find that unified memory support is currently not suitable for serving LLMs. First, it does not support partial freeing, preventing reclamation of physical memory of individual requests. Second, it lacks support for memory aliasing which prevents de-duplication of KV cache content in physical memory (de-duplication is useful when requests share a common prefix [51]), consequently limiting batch size. `cudaMallocManaged` also allocates 2MB pages by default which can cause severe fragmentation. However, our changes in NVIDIA drivers are based on unified memory: we allocate virtual tensors using `cudaMallocManaged`, enabling support for partial freeing and page sharing with additional APIs. In addition, we introduce latency hiding optimizations and support for smaller pages. Therefore, one could consider our extensions to NVIDIA drivers as "unified memory optimized for LLM serving".

8.2 Reducing Fragmentation via Tensor Slicing

To reduce fragmentation caused by 2MB pages, we also provide an alternative method that does not require modifying NVIDIA drivers. In this method, we use a single 2MB page to store the KV cache tokens of all layers for a given request. This can be done by allocating one virtual tensor of shape $[B, L, N, H, D]$ for K cache (and one for V cache) and slicing it across all layers, instead of allocating $2 \times N$ virtual tensors of shape $[B, L, H, D]$. This reduces fragmentation to $1/N$ of the

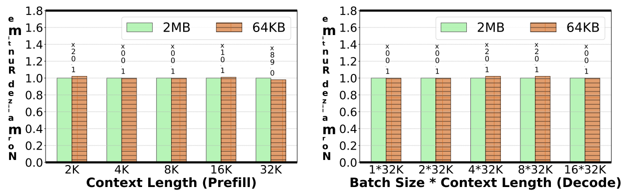


图 14. 页面大小对 FlashAttention-2 预填充（左）和解码（右）注意内核运行时的影响（型号：Llama-3-8B）。

Model	Block Size (# Tokens in a page-group)			
	64KB	128KB	256KB	2MB
Yi-6B (TP-1)	64	128	256	2048
Yi-6B (TP-2)	128	256	512	4096
Llama-3-8B (TP-1)	32	64	128	1024
Llama-3-8B (TP-2)	64	128	256	2048
Yi-34B (TP-1)	32	64	128	1024
Yi-34B (TP-2)	64	128	256	2048

表 8. KV 缓存块大小与页组大小和张量并行度的函数关系。

与使用 2MB 页面相比，当使用 64KB 页面分配 KV 缓存时，注意力内核的执行延迟在很大程度上不受影响。在另一个实验中，我们发现这些结果也与非常大的模型一致，例如 Llama-3-70B 和 GPT-3-175B。我们将此归因于注意算子的常规计算模式以及明确尝试避免不规则内存访问的手动调整实现。

表 8 显示了块大小，即每个物理页组的令牌数量。较小的页组可实现细粒度分配，接近 vLLM 建议的 16-32 块大小（FlashAttention-2 中的最小块大小为 256 – 高于 vAttention）。这表明 vAttention 在减少碎片方面与 PagedAttention 一样有效。就端到端系统性能而言，我们发现 2MB 页面对于许多在线服务场景来说已经足够了，在这些场景中，延迟限制阻止使用非常高的批量大小。相反，较小的页组更适合以吞吐量为导向的场景，在这种场景中，最大化批处理大小对于获得峰值性能非常重要。例如，使用 2MB 页面可以在动态跟踪上为 Yi-6B、Llama-3-8B 和 Yi-34B 提供高达 187、203 和 56 的批量大小（数据集：OpenChat [74]，负载：每秒 7 个查询）。相比之下，64KB 页面有助于为我们的模型提供高达 240、258 和 68 的批量大小，如图 15 所示。

7.6.4 内存分配带宽。表 9 显示，即使使用 64KB 页面（我们最小的页面），vAttention 也可以为每个 GPU 每秒分配多达 7.6GB 物理内存。这比每秒 750MB 的最大内存分配率高出一个数量级以上。

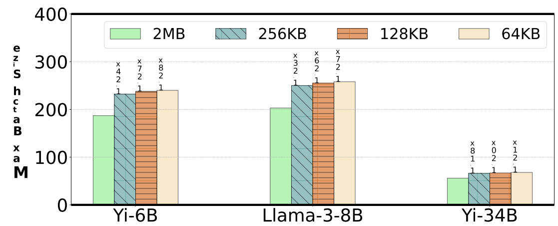


图 15. 针对动态工作负载使用不同页组大小获得的最大批处理大小。

Config.	64KB	128KB	256KB	2MB
TP-1	7.59	14.56	27.04	35.17
TP-2	15.18	29.12	54.08	70.34

表 9. 具有不同分配粒度的物理内存分配带宽（GB 每秒）。

解码（图 4）。较大的页组大小和较高的 TP 维度会成比例地增加内存分配率。因此，CUDA VMM API 的内存分配带宽对于 LLM 推理来说绰绰有余。

8 讨论

8.1 通过统一内存管理KV缓存

为了实现动态内存分配，我们还考虑通过 cudaMallocManaged [58, 60] 利用统一内存。然而，我们发现统一内存支持目前不适合服务于LLM。首先，它不支持部分释放，从而防止回收单个请求的物理内存。其次，它缺乏对内存别名的支持，内存别名会阻止物理内存中 KV 缓存内容的重复数据删除（当请求共享公共前缀时重复数据删除很有用 [51]），从而限制批量大小。 cudaMallocManaged 还默认分配 2MB 页面，这可能会导致严重的碎片。然而，我们对 NVIDIA 驱动程序的更改基于统一内存：我们使用 cudaMallocManaged 分配虚拟张量，从而支持部分释放和与其他 API 的页面共享。此外，我们还引入了延迟隐藏优化和对较小页面的支持。因此，人们可以将我们对 NVIDIA 驱动程序的扩展视为“针对 LLM 服务优化的统一内存”。

8.2 通过张量切片减少碎片

为了减少 2MB 页面造成的碎片，我们还提供了一种不需要修改 NVIDIA 驱动程序的替代方法。在此方法中，我们使用单个 2MB 页面来存储给定请求的所有层的 KV 缓存令牌。这可以通过为 K 缓存分配一个形状为 $[B, L, N, H, D]$ 的虚拟张量（以及为 V 缓存分配一个）并将其跨所有层进行切片来完成，而不是分配 2 个形状为 $[B, L, H, D]$ 的虚拟张量。这将碎片减少到 $1/N$

Model	Block size (# Tokens in a 2MB page) w/o Tensor Slicing	w/ Tensor Slicing
Yi-6B (TP-1)	2048	64
Yi-6B (TP-2)	4096	128
Llama-3-8B (TP-1)	1024	32
Llama-3-8B (TP-2)	2048	64
Yi-34B (TP-1)	1024	18
Yi-34B (TP-2)	2048	36

Table 10. KV cache block size with and without tensor slicing with 2MB pages.

```
-import flash_attn as fa
+import flashinfer as fi

-def flash_attn_prefill(q, k_cache, v_cache, kv_len):
-    k = k_cache[:, :kv_len, :, :]
-    v = v_cache[:, :kv_len, :, :]
-    return fa.flash_attn_func(q, k, v, causal=True)
+def flash_infer_prefill(q, k_cache, v_cache, kv_len):
+    q = q.squeeze(0)
+    k = k_cache.squeeze(0)[:kv_len, :, :]
+    v = v_cache.squeeze(0)[:kv_len, :, :]
+    return fi.single_prefill_with_kv_cache(q, k, v, causal=True)
```

Figure 16. Illustration of code changes needed to replace the prefill attention kernel of FlashAttention-2 by FlashInfer when using vAttention for memory allocation.

prior design as shown in Table 10. In this design, the K cache (or V cache) of a particular layer n is represented by slice $[B, L, n : n+1, H, D]$ of the original tensor, and can be passed to attention kernels for computation. However, note that with tensor slicing, the K cache (and V cache) of each layer is no longer contiguous. Computing attention over such a tensor slice is possible only if the the attention kernel supports memory addressing with strides. While many kernels (e.g., FlashAttention-2) support strides out-of-the-box, the earlier versions of FlashInfer lacked such support [14]. Therefore, relying on tensor slicing as the primary method of reducing fragmentation would have prevented vAttention from supporting FlashInfer kernels. Hence, to support unmodified FlashInfer kernels, we chose to add support for smaller pages in NVIDIA drivers. Importantly, our two solutions are compatible with each other; one could deploy smaller pages with tensor slicing to further reduce fragmentation, if required.

8.3 Programming Effort

PagedAttention requires significant programming effort. For example, integrating FlashInfer decode kernels in vLLM required more than 600 lines of code changes in over 15 files [24, 27, 30]. Implementing the initial paging support in FlashAttention-2 kernel also required ≈ 280 lines of code changes [20] and additional efforts to support small block sizes [8]. In contrast, vAttention makes it feasible to replace one attention kernel with another with only a few lines of code changes in the serving framework (Figure 16). Note that the example shown in Figure 16 has no interaction with memory management; this is precisely the goal of vAttention, i.e., when a slow kernel is replaced by a fast kernel, memory management should continue to work transparently.

9 Related Work

Optimizing LLM inference is an active area of research [44, 50, 53–57, 71, 72, 81, 83]. Various techniques have been proposed to improve different aspects of LLM serving like batching [35, 51, 78], disaggregation [48, 63, 82], scheduling [69, 75]. However, central to all these techniques is the need for efficient KV cache memory management. Since vLLM, PagedAttention has been adopted in various serving frameworks e.g., TensorRT-LLM [6], LightLLM [4], and libraries e.g., FlashAttention-2 [1] and FlashInfer [3]. Some other concurrent works have also proposed optimizations for attention kernels [29, 39, 47, 49, 66, 67, 76, 80]. Deploying new kernels for inference is hard with PagedAttention. vAttention offers an alternate – which we believe is also a more principled – approach to dynamic KV cache memory management that makes it easier to deploy new kernels.

In a recent work, GMLake [45] showed that using CUDA virtual memory support can mitigate fragmentation in DNN training jobs, increasing training batch size. In particular, GMLake uses CUDA support to coalesce multiple smaller physical memory pages into a single virtually contiguous object that can prevent out-of-memory errors. In contrast, vAttention targets optimizing inference workloads.

10 Conclusion

In this paper, we propose vAttention for dynamic KV cache memory management in LLM serving systems. The key highlight of vAttention is that it mitigates fragmentation in physical memory while retaining the contiguity of KV cache in virtual memory. We present various examples to highlight that vAttention reduces programming burden while improving portability and performance compared to the popular PagedAttention approach.

Acknowledgments

We thank our shepherd Sam Ainsworth, along with the anonymous ASPLOS reviewers, for their valuable feedback. We thank NVIDIA for useful discussions on CUDA VMM APIs and for sharing their experiences with vAttention. We also thank members of the open-source community (GitHub user names izhuhaoran and tongping) for trying to upstream the vAttention approach to vLLM [23, 31], and Zihao Ye for clarifying the role of register spilling in the runtime overhead of PagedAttention. Ajay Nayak is supported by the Prime Minister's Fellowship Scheme for Doctoral Research, co-sponsored by the Confederation of Indian Industry, the Government of India, and Microsoft Research India.

Model	Block size (# Tokens in a 2MB page) w/o Tensor Slicing	w/ Tensor Slicing
Yi-6B (TP-1)	2048	64
Yi-6B (TP-2)	4096	128
Llama-3-8B (TP-1)	1024	32
Llama-3-8B (TP-2)	2048	64
Yi-34B (TP-1)	1024	18
Yi-34B (TP-2)	2048	36

表 10. 使用和不使用 2MB 页面张量切片的 KV 缓存块大小。

```
-import flash_attn as fa
+import flashinfer as fi

-def flash_attn_prefill(q, k_cache, v_cache, kv_len):
-    k = k_cache[:, :kv_len, :, :]
-    v = v_cache[:, :kv_len, :, :]
-    return fa.flash_attn_func(q, k, v, causal=True)
+def flash_infer_prefill(q, k_cache, v_cache, kv_len):
+    q = q.squeeze(0)
+    k = k_cache.squeeze(0)[:kv_len, :, :]
+    v = v_cache.squeeze(0)[:kv_len, :, :]
+    return fi.single_prefill_with_kv_cache(q, k, v, causal=True)
```

图 16. 使用 vAttention 进行内存分配时，用 FlashInfer 替换 FlashAttention-2 的预填充注意力内核所需的代码更改图示。

现有设计如表 10 所示。在该设计中，特定层 n 的 K 缓存（或 V 缓存）由原始张量的切片 $[B, L, n:n+1, H, D]$ 表示，并且可以传递到注意力内核进行计算。但请注意，使用张量切片时，每层的 K 缓存（和 V 缓存）不再连续。仅当注意力内核支持跨步内存寻址时，才可以在这样的张量切片上计算注意力。虽然许多内核（例如 FlashAttention-2）支持开箱即用的步幅，但早期版本的 FlashInfer 缺乏这种支持 [14]。因此，依靠张量切片作为减少碎片的主要方法会阻止 vAttention 支持 FlashInfer 内核。因此，为了支持未修改的 FlashInfer 内核，我们选择在 NVIDIA 驱动程序中添加对较小页面的支持。重要的是，我们的两种解决方案彼此兼容；如果需要，可以使用张量切片部署较小的页面，以进一步减少碎片。

8.3 编程工作量

PagedAttention 需要大量的编程工作。例如，在 vLLM 中集成 FlashInfer 解码内核需要在超过 15 个文件中进行 600 多行代码更改 [24,27,30]。在 FlashAttention-2 内核中实现初始分页支持还需要 ≈ 280 行代码更改 [20] 以及支持小块大小的额外工作 [8]。相比之下，vAttention 使得只需在服务框架中更改几行代码即可将一个注意力内核替换为另一个注意力内核（图 16）。请注意，图 16 中所示的示例与内存管理没有交互；这正是 vAttention 的目标，即当慢速内核被快速内核取代时，内存管理应该继续透明地工作。

9 相关工作

优化 LLM 推理是一个活跃的研究领域 [44,50,53–57,71,72,81,83]。人们提出了各种技术来改进LLM服务的不同方面，例如批处理[35,51,78]、分解[48,63,82]、调度[69,75]。然而，所有这些技术的核心是需要高效的 KV 缓存管理。自从vLLM以来，PagedAttention 已在各种服务框架（例如 TensorRT-LLM [6]、LightLLM [4]）和库（例如 FlashAttention-2 [1] 和 FlashInfer [3]）中采用。其他一些同时进行的工作也提出了注意力内核的优化[29,39,47,49,66,67,76,80]。使用 PagedAttention 部署新的推理内核很困难。 vAttention 提供了另一种动态 KV 缓存管理方法（我们认为这也是一种更有原则的方法），可以更轻松地部署新内核。

在最近的一项工作中，GMLake [45] 表明，使用 CUDA 虚拟内存支持可以减轻 DNN 训练作业中的碎片，从而增加训练批量大小。特别是，GMLake 使用 CUDA 支持将多个较小的物理内存页面合并为一个虚拟连续的对象，从而可以防止内存不足错误。相比之下，vAttention 的目标是优化推理工作负载。

10 结论

在本文中，我们提出了 vAttention 用于 LLM 服务系统中的动态 KV 缓存管理。 vAttention 的关键亮点在于它减少了物理内存中的碎片，同时保留了虚拟内存中 KV 缓存的连续性。我们提供了各种示例来强调，与流行的 PagedAttention 方法相比，vAttention 减少了编程负担，同时提高了可移植性和性能。

致谢

我们感谢我们的牧羊人 Sam Ainsworth 以及匿名 ASPLOS 审阅者提供的宝贵反馈。我们感谢 NVIDIA 对 CUDA VMM API 进行了有益的讨论，并分享了他们在 vAttention 方面的经验。我们还感谢开源社区的成员（GitHub 用户名 izhuhaoran 和 tongping）尝试将 vAttention 方法上游到 vLLM [23, 31]，并感谢 Zihao Ye 阐明了寄存器溢出在 PagedAttention 运行时开销中的作用。 Ajay Nayak 得到总理博士研究奖学金计划的支持，该计划由印度工业联合会、印度政府和微软印度研究院共同赞助。

References

[1] 2022. FlashAttention. https://github.com/Dao-AILab/flash-attention.
[2] 2023. Flash-Decoding for long-context inference. https://crfm.stanford.edu/2023/10/12/flashdecoding.html.
[3] 2023. FlashInfer: Kernel Library for LLM Serving. https://github.com/flashinfer-ai/flashinfer.
[4] 2023. LightLLM: A Light and Fast Inference Service for LLM. https://github.com/ModelTC/lightllm.
[5] 2023. Performance decay when using paged attention. https://github.com/NVIDIA/TensorRT-LLM/issues/75.
[6] 2023. TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. https://github.com/NVIDIA/TensorRT-LLM.
[7] 2023. Use optimized kernels for MQA/GQA. https://github.com/vllm-project/vllm/issues/1880.
[8] 2024. Add support for small page sizes. https://github.com/Dao-AILab/flash-attention/pull/824.
[9] 2024. Amazon CodeWhisperer. https://aws.amazon.com/codewhisperer/.
[10] 2024. Bing AI. https://www.bing.com/chat.
[11] 2024. ccdv/arxiv-summarization. https://huggingface.co/datasets/ccdv/arxiv-summarization.
[12] 2024. CUDA Toolkit Documentation: Virtual Memory Management. https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__VA.html.
[13] 2024. Custom CUDA kernels for KV cache copy operations. https://github.com/vllm-project/vllm/blob/main/csrc/cache_kernels.cu.
[14] 2024. Custom strides to support non-contiguous KV cache. https://github.com/flashinfer-ai/flashinfer/commit/85b1878996a29814f674ee5000facb1e2e763d9a.
[15] 2024. Faster Transformer. https://github.com/NVIDIA/FasterTransformer.
[16] 2024. [Feature]: FlashAttention 3 support. https://github.com/vllm-project/vllm/issues/6348#issuecomment-2540969988.
[17] 2024. Fix eager mode performance. https://github.com/vllm-project/vllm/pull/2377.
[18] 2024. Github Copilot. https://github.com/features/copilot.
[19] 2024. Google Bard. https://bard.google.com.
[20] 2024. Implement Page KV Cache. https://github.com/Dao-AILab/flash-attention/commit/54e80a3829c6d2337570d01e78ebd9529c02d342.
[21] 2024. Meta-Llama-3-8B. https://huggingface.co/meta-llama/Meta-Llama-3-8B.
[22] 2024. Pascal MMU Format Changes. https://nvidia.github.io/open-gpu-doc/pascal/gp100-mmu-format.pdf.
[23] 2024. PoC of dAttention support (based on vAttention). https://github.com/vllm-project/vllm/pull/9078.
[24] 2024. Refactor Attention Take 2. https://github.com/vllm-project/vllm/pull/3462.
[25] 2024. Replit Ghostwriter. https://replit.com/site/ghostwriter.
[26] 2024. [Roadmap] vLLM Roadmap Q4 2024 #9006. https://github.com/vllm-project/vllm/issues/9006#issue-2559831134.
[27] 2024. Separate attention backends. https://github.com/vllm-project/vllm/pull/3005/.
[28] 2024. Text Generation Inference. https://huggingface.co/text-generation-inference.
[29] 2024. Tile primitives for speedy kernels. https://github.com/HazyResearch/ThunderKittens.
[30] 2024. Use FlashInfer for Decoding. https://github.com/vllm-project/vllm/pull/4353.
[31] 2024. VMM KV cache for NVIDIA GPUs. https://github.com/vllm-project/vllm/pull/6102.
[32] 2024. Yi-34B-200K. https://huggingface.co/01-ai/Yi-34B-200K.
[33] 2024. Yi-6B-200K. https://huggingface.co/01-ai/Yi-6B-200K.
[34] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework For LLM Inference. *Proceedings of The Seventh Annual Conference on Machine Learning and Systems, 2024, Santa Clara* (2024).
[35] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 117–134. https://www.usenix.org/conference/osdi24/presentation/agrawal
[36] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 [cs.LG]
[37] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245 [cs.CL]
[38] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv:2004.05150 [cs.CL]
[39] Ganesh Bikshandi and Jay Shah. 2023. A Case Study in CUDA Kernel Fusion: Implementing FlashAttention-2 on NVIDIA Hopper Architecture using the CUTLASS Library. arXiv:2312.11918 [cs.LG]
[40] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. arXiv:1904.10509 [cs.LG]
[41] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. PaLM: Scaling Language Modeling with Pathways. *CoRR* abs/2204.02311 (2022). https://doi.org/10.48550/arXiv.2204.02311 arXiv:2204.02311
[42] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 [cs.LG]
[43] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2024. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1189, 16 pages.
[44] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung, and Ion Stoica. 2024. Mélange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity. arXiv:2404.14527 [cs.DC] https://arxiv.org/abs/2404.14527
[45] Cong Guo, Rui Zhang, Jiale Xu, Jingwen Leng, Zihan Liu, Ziyu Huang, Minyi Guo, Hao Wu, Shouren Zhao, Junping Zhao, and Ke Zhang. 2024. GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 450–466. https://doi.org/10.1145/3620665.3640423

参考

[1]2022。FlashAttention。 https://github.com/Dao-AILab/flash-attention 。 [2] 2023。用于长上下文推理的 Flash 解码。 https://crfm.斯坦福。 edu/2023/10/12/flashdecoding.html。 [3]2023。FlashInfer：LLM 服务的内核库。 https://github.com/flashinfer-ai/flashinfer。 [4] 2023. LightLLM：LLM 的轻型快速推理服务。 https://github.com/ModelTC/lightllm。 [5] 2023。使用分页注意力时性能下降。 https://github。 com/NVIDIA/TensorRT-LLM/issues/75。 [6]2023。TensorRT-LLM：用于优化大型语言模型推理的 TensorRT 工具箱。 https://github.com/NVIDIA/TensorRT-LLM。 [7] 2023 年。使用 MQA/GQA 优化内核。 https://github.com/vllm-project/vllm/issues/1880。 [8] 2024。添加对小页面尺寸的支持。 https://github.com/Dao-AILab/flash-attention/pull/824。 [9]2024。亚马逊 CodeWhisperer。 https://aws.amazon.com/codewhisperer/。 [10]2024.必应人工智能. https://www.bing.com/chat。 [11] 2024.ccdv/arxiv-摘要。 https://huggingface.co/datasets/ccdv/arxiv-summarization。 [12]2024。CUDA 工具包文档：虚拟内存管理。 https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA_ _VA.html。 [13] 2024 年。用于 KV 缓存复制操作的自定义 CUDA 内核。 https://github.com/vllm-project/vllm/blob/main/csrc/cache_kernels.cu。 [14] 2024。支持非连续 KV 缓存的自定义步骤。 https://github.com/flashinfer-ai/flashinfer/commit/85b1878996a29814f674ee5000facb1e2e763d9a。 [15]2024。更快的变压器。 https://github.com/NVIDIA/FasterTransformer。 [16]2024。[功能]：FlashAttention 3 支持。 https://github.com/vllm-project/vllm/issues/6348#issuecomment-2540969988。 [17] 2024 年。修复 eager 模式性能。 https://github.com/vllm-project/vllm/pull/2377。 [18]2024。Github 副驾驶。 https://github.com/features/copilot。 [19]2024。谷歌吟游诗人。 https://bard.google.com。 [20]2024.实现页面KV缓存。 https://github.com/Dao-AILab/flash-attention/commit/54e80a3829c6d2337570d01e78ebd9529c02d342。 [21]2024。Meta-Llama-3-8B。 https://huggingface.co/meta-llama/Meta-Llama-3-8B 。 [22] 2024 年。Pascal MMU 格式更改。 https://nvidia.github.io/open-gpu-doc/pascal/gp100-mmu-format.pdf。 [23] 2024. dAttention 支持的 PoC（基于 vAttention）。 https://github。 com/vllm-project/vllm/pull/9078 。 [24] 2024 年。重构注意事项 2。https://github.com/vllm-project/vllm/pull/3462。 [25] 2024. Replit Ghostwriter。 https://replit.com/site/ghostwriter。 [26] 2024 年。[路线图] vLLM 路线图 2024 年第 4 季度 #9006。 https://github.com/vllm-project/vllm/issues/9006#issue-2559831134。 [27] 2024。单独的注意力后端。 https://github.com/vllm-project/vllm/pull/3005/。 [28]2024。文本生成推理。 https://huggingface.co/text- Generation-inference。 [29] 2024。快速内核的平铺原语。 https://github.com/HazyResearch/ThunderKittens。 [30]2024。使用FlashInfer进行解码。 https://github.com/vllm-project/vllm/pull/4353。 [31] 2024 年。NVIDIA GPU 的 VMM KV 缓存。 https://github.com/vllm-project/vllm/pull/6102。 [32]2024。Yi-34B-200K。 https://huggingface.co/01-ai/Yi-34B-200K。 [33] 2024. Yi-6B-200K。 https://huggingface.co/01-ai/Yi-6B-200K。 [34] Amey Agrawal、Nitin Kedia、Jayashree Mohan、Ashish Panwar、Nipun Kwatra、Bhargav S Gulavani、Ramachandran Ramjee 和 Alexey Tumanov 。 2024. Vidur：大语言模型推理模拟框架推理。第七届机器学习和系统年会论文集，2024 年，圣克拉拉（2024 年）。 [35] Amey Agrawal、Nitin Kedia、Ashish Panwar、Jayashree Mohan、Nipun Kwatra、Bhargav Gulavani、Alexey Tumanov 和 Ramachandran Ramjee。 2024. 使用 Sarathi-Serve 控制 LLM 推理中的吞吐量与延迟权衡。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。 USENIX 协会，加利福尼亚州圣克拉拉，117–134。 https://www.usenix.org/conference/osdi24/presentation/agrawal [36] Amey Agrawal、Ashish Panwar、Jaashree Mohan、Nipun Kwatra、Bhargav S. Gulavani 和 Ramachandran Ramjee。 2023.SARATHI：通过分块预填充的搭载解码进行高效的 LLM 推理。 arXiv:2308.16369 [cs.LG] [37] Joshua Ainslie、James Lee-Thorp、Michiel de Jong、Yury Zemlyanskiy、Federico Lebrón 和 Sumit Sanghai。 2023.GQA：从多头检查点训练通用多查询变压器模型。 arXiv:2305.13245 [cs.CL] [38] Iz Beltagy、Matthew E. Peters 和 Arman Cohan。 2020.Longformer：长文档 Transformer。 arXiv:2004.05150 [cs.CL] [39] Ganesh Bikshandi 和 Jay Shah。 2023. CUDA 内核融合案例研究：使用 CUTLASS 库在 NVIDIA Hopper 架构上实现 FlashAttention-2。 arXiv:2312.11918 [cs.LG] [40] Rewon Child、Scott Gray、Alec Radford 和 Ilya Sutskever。 2019。用稀疏变压器生成长序列。 arXiv:1904.10509 [cs.LG] [41] Aakanksha Chowdhery、Sharan Narang、Jacob Devlin、Maarten Bosma、Gaurav Mishra、Adam Roberts、Paul Barham、Hyung Won Chung、Charles Sutton、Sebastian Gehrmann、Parker Schuh、Kensen Shi、Sasha Tsvyashchenko、Joshua Maynez、Abhishek Rao、帕克·巴恩斯, 伊·泰, 诺姆·沙泽尔, 维诺库玛·普拉巴卡兰, 艾米丽·莱夫, Nan Du, 本·哈钦森, 雷纳·波普, 詹姆斯·布拉德伯里, 雅各布·奥斯汀, 迈克尔·伊萨德, 盖·古尔-阿里, 尹鹏城, Toju Duke, 安塞姆·列夫斯卡亚, 桑杰·格马瓦特, 苏尼帕·戴夫, 亨利克·米查勒夫斯基, 泽维尔·加西亚, 韦丹特·米斯拉, 凯文·罗宾逊, Liam Fedus、Denny Zhou、Daphne Ippolito、David Luan、Hyeontaek Lim、Barret Zoph、Alexander Spiridonov、Ryan Sepassi、David Dohan、Shivani Agrawal、Mark Omernick、Andrew M. Dai、Thanumalayan Sankaranarayana Pillai、Marie Pellat、Aitor Lewkowycz、Erica Moreira、Rewon Child、Oleksandr Polozov、Katherine Lee、周宗伟、王学智、Brennan Saeta、Mark Diaz、Orhan Firat、Michele Catasta、Jason Wei、Kathy Meier-Hellstern、Douglas Eck、Jeff Dean、Slav Petrov 和 Noah Fiedel。 2022.PaLM：通过路径扩展语言建模。 CoRR 绝对/2204.02311 (2022)。 https://doi.org/10.48550/arXiv.2204.02311 arXiv:2204.02311 [42] 三道。 2023.FlashAttention-2：更快的注意力，更好的并行性和工作分区。 arXiv:2307.08691 [cs.LG] [43] Tri Dao、Daniel Y. Fu、Stefano Ermon、Atri Rudra 和 Christopher Ré。 2024. FLASHATTENTION：具有 IO 意识的快速、内存高效的精确注意力。第 36 届神经信息处理系统国际会议（美国路易斯安那州新奥尔良）（NIPS '22）论文集。 Curran Associates Inc.，美国纽约州雷德胡克，第 1189 条，16 页。 [44] Tyler Griggs、Xiaoxuan Liu、Jiayang Yu、Doyoung Kim、Wei-Lin Chiang、Alvin Cheung 和 Ion Stoica。 2024. Mélange：利用 GPU 异构性提供成本高效的大型语言模型服务。 arXiv:2404.14527 [cs.DC] https://arxiv.org/abs/2404.14527 [45] 郭聪、张锐、徐佳乐、冷静文、刘子涵、黄子宇、郭敏仪、吴浩、赵守仁、赵俊平和张科。 2024.GMLake：使用虚拟内存拼接进行大规模 DNN 训练的高效且透明的 GPU 内存碎片整理。第 29 届 ACM 国际编程语言和操作系统架构支持会议论文集，第 2 卷（美国加利福尼亚州拉荷亚）(ASPLOS ’24)。计算机协会，美国纽约州纽约市，450–466。 https://doi.org/10.1145/3620665.3640423

[46] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. arXiv:2401.08671 [cs.PF]

[47] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, kangdi chen, Yuhan Dong, and Yu Wang. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 148–161. https://proceedings.mlsys.org/paper_files/paper/2024/file/5321b1dabcd2be188d796c21b733e8c7-Paper-Conference.pdf

[48] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. 2024. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads. *arXiv preprint arXiv:2401.11181* (2024).

[49] Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2024. POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. arXiv:2410.18038 [cs.LG] https://arxiv.org/abs/2410.18038

[50] Ferdi Kossmann, Bruce Fontaine, Daya Khudia, Michael Cafarella, and Samuel Madden. 2024. Is the GPU Half-Empty or Half-Full? Practical Scheduling Techniques for LLMs. arXiv:2410.17840 [cs.LG] https://arxiv.org/abs/2410.17840

[51] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. https://doi.org/10.1145/3600006.3613165

[52] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. 2016. Coordinated and Efficient Huge Page Management with Ingens. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 705–721. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/kwon

[53] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. arXiv:2406.19707 [cs.LG] https://arxiv.org/abs/2406.19707

[54] Da Ma, Lu Chen, Situo Zhang, Yuxun Miao, Su Zhu, Zhi Chen, Hongshen Xu, Hanqi Li, Shuai Fan, Lei Pan, and Kai Yu. 2024. Compressing KV Cache for Long-Context LLM Inference with Inter-Layer Attention Similarity. arXiv:2412.02252 [cs.CL] https://arxiv.org/abs/2412.02252

[55] Saaduddin Mahmud, Mason Nakamura, and Shlomo Zilberstein. 2024. MAPLE: A Framework for Active Preference Learning Guided by Large Language Models. arXiv:2412.07207 [cs.LG] https://arxiv.org/abs/2412.07207

[56] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2024. Helix: Distributed Serving of Large Language Models via Max-Flow on Heterogeneous GPUs. arXiv:2406.01566 [cs.LG] https://arxiv.org/abs/2406.01566

[57] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2023. SpotServe: Serving Generative Large Language Models on Preemptible Instances. arXiv:2311.15566 [cs.DC] https://arxiv.org/abs/2311.15566

[58] Ajay Nayak, Pratheek B., Vinod Ganapathy, and Arkaprava Basu. 2021. (Mis)managed: A Novel TLB-based Covert Channel on GPUs. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security* (Virtual Event, Hong Kong) *(ASIA CCS '21)*. Association for Computing Machinery, New York, NY, USA, 872–885. https://doi.org/10.1145/3433210.3453077

[59] Matthew Nicely and NVIDIA. 2024. Accelerating Transformers with NVIDIA cuDNN 9. https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/

[60] NVIDIA. 2024. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[61] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). https://doi.org/10.48550/arXiv.2303.08774 arXiv:2303.08774

[62] Ashish Panwar, Sorav Bansal, and K. Gopinath. 2019. HawkEye: Efficient Fine-Grained OS Support for Huge Pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 347–360. https://doi.org/10.1145/3297858.3304064

[63] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. https://doi.org/10.1109/ISCA59077.2024.00019

[64] Bharath Pichai, Lisa Hsu, and Abhishek Bhattacharjee. 2014. Architectural support for address translation on GPUs: designing memory management units for CPU/GPUs with unified address spaces. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems* (Salt Lake City, Utah, USA) *(ASPLOS '14)*. Association for Computing Machinery, New York, NY, USA, 743–758. https://doi.org/10.1145/2541940.2541942

[65] B Pratheek, Neha Jawalkar, and Arkaprava Basu. 2022. Designing Virtual Memory System of MCM GPUs. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 404–422. https://doi.org/10.1109/MICRO56248.2022.00036

[66] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. 2024. Lean Attention: Hardware-Aware Scalable Attention Mechanism for the Decode-Phase of Transformers. arXiv:2405.10480 [cs.AR] https://arxiv.org/abs/2405.10480

[67] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. (2024).

[68] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150 [cs.NE]

[69] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: high-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) *(ICML'23)*. JMLR.org, Article 1288, 23 pages.

[70] Seunghee Shin, Michael LeBeane, Yan Solihin, and Arkaprava Basu. 2018. Neighborhood-Aware Address Translation for Irregular GPU Applications. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 352–363. https://doi.org/10.1109/MICRO.2018.00036

[71] Prajwal Singhania, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. 2024. Loki: Low-rank Keys for Efficient Sparse Attention. arXiv:2406.02542 [cs.LG] https://arxiv.org/abs/2406.02542

[72] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving. arXiv:2403.01876 [cs.DC] https://arxiv.org/abs/2403.01876

[73] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[74] Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. 2023. OpenChat: Advancing Open-source Language Models

[46] Connor Holmes、Masahiro Tanaka、Michael Wyatt、Ammar Ahmad Awan、Jeff Rasley、Samyam Rajbhandari、Reza Yazdani Aminabadi、Heyangqin、Arash Bakhtiari、Lev Kurilenko 和 Yuxiong He。 2024. DeepSpeed-FastGen：通过 MII 和 DeepSpeed-Inference 为大语言模型成高通量文本。 arXiv:2401.08671 [cs.PF] [47] Ke Hong、Guohao Dai、Jiaming Xu、Qiuli Mao、Xiuhong Li、Jun Liu、kangdi chen、Yuhan Dong 和 Yu Wang。 2024. FlashDecoding++：通过异步、扁平 GEMM 优化和启发式进行更快的大型语言模型推理。在《机器学习和系统学报》中，P. Gibbons、G. Pekhimenko 和 C. De Sa（编辑），卷。 6. 148-161。 https://proceedings.mlsys.org/paper_files/paper/2024/file/5321b1dabcd2be188d796c21b733e8c7-Paper-Conference.pdf [48] Cunchen Hu，Heyang Huang，Liangliang Xu，Xusheng Chen，Jiang Xu，Shuang Chen，Hao Feng，Chenxi Wang，Sa Wang，Yungang Bao，等。 2024. 无干扰推理：针对混合下游工作负载分解 LLM 推理。 arXiv 预印本 arXiv:2401.11181 (2024)。 [49] Aditya K Kamath、Ramya Prabhu、Jayashree Mohan、Simon Peter、Ramachandran Ramjee 和 Ashish Panwar。 2024. POD-Attention：解锁完整的预填充解码重叠以实现更快的 LLM 推理。 arXiv:2410.18038 [cs.LG] https://arxiv.org/abs/2410.18038 [50] Ferdi Kossmann、Bruce Fontaine、Daya Khudia、Michael Cafarella 和 Samuel Madden。 2024. GPU是半空还是半满？大语言模型的日程安排技术。 arXiv:2410.17840 [cs.LG] https://arxiv.org/abs/2410.17840 [51] Woosuk Kwon、Zhuohan Li、Siyuan Zhuang、Ying Shen、Lianmin Cheng、Codyhao Yu、Joseph Gonzalez、Hao Zhu 和 Ion Stoica。 2023.使用 PagedAttention 进行大型语言模型的高效内存管理。第 29 届操作系统原理研讨会论文集（德国科布伦茨）(SOSP '23)。计算机协会，美国纽约州纽约市，611-626。 https://doi.org/10.1145/3600006.3613165 [52] Youngjin Kwon、Hangchen Yu、Simon Peter、Christopher J. Rossbach 和 Emmett Witchel。 2016. 与 Ingens 协调高效的大页面管理。第 12 届 USENIX 操作系统设计与实现研讨会 (OSDI 16)。 USENIX 协会，萨凡纳，佐治亚州，705–721。 https://www.usenix.org/conference/osdi16/technical-sessions/presentation/kwon [53] Wonbeom Lee、Jungi Lee、Junghwan Seo 和 Jaewoong Sim。 2024. InfiniGen：利用动态 KV 缓存管理对大型语言模型进行高效生成推理。 arXiv:2406.19707 [cs.LG] https://arxiv.org/abs/2406.19707 [54] Da Ma，Lu Chen，Situo Zhang，Yuxun Miao，Su Zhu，Zhi Chen，Hong-shen Xu，Hanqi Li，Shuaai Fan，Lei Pan，and Kai Yu。 2024.压缩 KV 缓存以实现具有层间注意力相似性的长上下文 LLM 推理。 arXiv:2412.02252 [cs.CL] https://arxiv.org/abs/2412.02252 [55] Saaduddin Mahmud、Mason Nakamura 和 Shlomo Zilberstein。 2024. MAPLE：大型语言模型引导的主动偏好学习框架。 arXiv:2412.07207 [cs.LG] https://arxiv.org/abs/2412.07207 [56] Yixuan Mei、Yonghao Zhuang、Xupeng Miao、Jun Cheng Yang、Zhihao Jia 和 Rashmi Vinayak。 2024. Helix：通过异构 GPU 上的 Max-Flow 实现大型语言模型的分布式服务。 arXiv:2406.01566 [cs.LG] https://arxiv.org/abs/2406.01566 [57] Xupeng Miao、Chuanan Shi、Jiangfei Duan、Xiaoli Xi、Dahua Lin、Bin Cui 和zhihao Jia。 2023.SpotServe：在可抢占实例上提供生成大型语言模型。 arXiv:2311.15566 [cs.DC] https://arxiv.org/abs/2311.15566 [58] Ajay Nayak、Pratheek B.、Vinod Ganapathy 和 Arkaprava Basu。 2021。（错误）管理：GPU 上基于 TLB 的新型隐蔽通道。 2021 年 ACM 亚洲计算机和通信安全会议（虚拟活动，香港）（ASIA CCS '21）的会议记录。计算机协会，美国纽约州纽约市，872–885。 https://doi.org/10.1145/3433210.3453077

[59] 马修·尼斯利和 NVIDIA。 2024 年。使用 NVIDIA cuDNN 9 加速 Transformers。https://developer.nvidia.com/blog/acceleating-transformers-with-nvidia-cudnn-9/ [60] NVIDIA。 2024.CUDA C++ 编程指南。 https://docs.nvidia。 com/cuda/cuda-c-programming-guide/index.html [61] OpenAI。 2023.GPT-4 技术报告。 CoRR 绝对/2303.08774 (2023)。 https://doi.org/10.48550/arXiv.2303.08774 arXiv:2303.08774 [62] Ashish Panwar、Sorav Bansal 和 K. Gopinath。 2019.HawkEye：对大页面的高效细粒度操作系统支持。第二十四届编程语言和操作系统架构支持国际会议（美国罗德岛州普罗维登斯）(ASPLOS '19) 论文集。计算机协会，美国纽约州纽约市，347-360。 https://doi.org/10.1145/3297858.3304064 [63] Pratyush Patel、Esha Choukse、Chaojie Zhang、Aashaka Shah、Íñigo Goiri、Saeed Maleki 和 Ricardo Bianchini。 2024.Splitwise：使用相分裂的高效生成大语言模型 2024 年 ACM/IEEE 第 51 届计算机体系结构国际研讨会 (ISCA)。 118-132。 https://doi.org/10.1109/ISCA59077.2024.00019 [64] Bharath Pichai、Lisa Hsu 和 Abhishek Bhattacharjee。 2014. GPU 地址转换的架构支持：为具有统一地址空间的 CPU/GPU 设计内存管理单元。第 19 届编程语言和操作系统架构支持国际会议（美国犹他州盐湖城）(ASPLOS '14) 论文集。计算机协会，美国纽约州纽约市，743–758。 https://doi.org/10.1145/2541940.2541942 [65] B Pratheek、Neha Jawalkar 和 Arkaprava Basu。 2022.设计MCM GPU 的虚拟内存系统。 2022 年第 55 届 IEEE/ACM 国际微架构研讨会 (MICRO)。 404–422。 https://doi.org/10.1109/MICRO56248.2022.00036 [66] Rya Sanovar、Srikant Bharadwaj、Renee St. Amant、Victor Rühle 和 Saravan Rajmohan。 2024.精益注意力：变压器解码阶段的硬件感知可扩展注意力机制。 arXiv:2405.10480 [cs.AR] https://arxiv.org/abs/2405.10480 [67] Jay Shah、Ganesh Bikshandi、Ying Zhang、Vijay Thakkar、Pradeep Ramani 和 Tri Dao。 2024. FlashAttention-3：具有异步和低精度的快速准确的注意力。 （2024）。 [68] 诺姆·沙泽尔。 2019。快速变压器解码：一个写头即可满足您的需求。 arXiv:1911.02150 [cs.NE] [69] Ying Shen、Lianmin Cheng、Binhang Yuan、Zhuohan Li、Max Ryabinin、Beidi Chen、Percy Liang、Christopher Ré、Ion Stoica 和 Ce Zhang。 2023. FlexGen：使用单个 GPU 对大型语言模型进行高吞吐量生成推理。第 40 届国际机器学习会议（美国夏威夷檀香山）会议记录 (ICML'23)。 JMLR.org，第 1288 条，23 页。 [70] Seunghee Shin、Michael LeBeane、Yan Solihin 和 Arkaprava Basu。 2018. 不规则 GPU 应用的邻域感知地址转换。 2018 年第 51 届 IEEE/ACM 国际微架构研讨会（MICRO）。 352–363。 https://doi.org/10.1109/MICRO。 2018.00036 [71] Prajwal Singhania、Siddharth Singh、Shwai He、Soheil Feizi 和 Abhinav Bhatele。 2024.Loki：高效稀疏注意力的低秩键。 arXiv:2406.02542 [cs.LG] https://arxiv.org/abs/2406.02542 [72] Foteini Strati、Sara Mcallister、Amar Phanishayee、Jakub Tarnawski 和 Ana Klimovic。 2024. DéjàVu：用于快速、容错生成 LLM 服务的 KV 缓存流。 arXiv:2403.01876 [cs.DC] https: //arxiv.org/abs/2403.01876 [73] Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N Gomez、 ukasz Kaiser 和 Illia Polosukhin。 2017 年。您所需要的就是注意力。 《神经信息处理系统进展》，I. Guyon、U. Von Luxburg、S. Bengio、H. Wallach、R. Fergus、S. Vishwanathan 和 R. Garnett（编辑），卷。 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf [74] 王冠、程思杰、詹贤元、李贤刚、宋森和刘阳。 2023.OpenChat：推进开源语言模型

with Mixed-Quality Data. arXiv:2309.11235 [cs.CL]

[75] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920 [cs.LG]

[76] Mengdi Wu, Xinhao Cheng, Oded Padon, and Zhihao Jia. 2024. A Multi-Level Superoptimizer for Tensor Programs. arXiv:2405.05751

[77] Zihao Ye, Lequn Chen, Ruihang Lai, Yilong Zhao, Size Zheng, Junru Shao, Bohan Hou, Hongyi Jin, Yifei Zuo, Liangsheng Yin, Tianqi Chen, and Luis Ceze. 2024. Accelerating Self-Attentions for LLM Serving with FlashInfer. https://flashinfer.ai/2024/02/02/introduce-flashinfer.html

[78] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[79] Zhenkai Zhang, Tyler Allen, Fan Yao, Xing Gao, and Rong Ge. 2023. TunneLs for Bootlegging: Fully Reverse-Engineering GPU TLBs for Challenging Isolation Guarantees of NVIDIA MIG. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 960–974. https://doi.org/10.1145/3576915.3616672

[80] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 34661–34710. https://proceedings.neurips.cc/paper_files/paper/2023/file/6ceefa7b15572587b78ecfcebb2827f8-Paper-Conference.pdf

[81] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104

[82] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[83] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2024. NanoFlow: Towards Optimal Large Language Model Serving Throughput. arXiv:2408.12757 [cs.DC] https://arxiv.org/abs/2408.12757

具有混合质量的数据。 arXiv:2309.11235 [cs.CL] [75] Bingyang Wu、Yinminzhong、Zili Zhang、Gang Huang、Xuanzhe Liu 和 Xin Jin。 2023。大型语言模型的快速分布式推理服务。 arXiv:2305.05920 [cs.LG] [76]Mengdi Wu、Xinhao Cheng、Oded Padon 和zhihao Jia。 2024.张量程序的多级超级优化器。 arXiv:2405.05751 [77] Zihao Ye、Lequn Chen、Ruihang Lai、Yilong Zhao、Size Cheng、Junru Shao、Bohan Hou、hongyi Jin、Yifei Zuo、Liangsheng Yin、Tianqi Chen 和 Luis Ceze。 2024. 使用 FlashInfer 加速 LLM 服务的自我注意力。 https://flashinfer.ai/2024/02/02/introduce-flashinfer.html [78] Gyeong-In Yu、Joo Seong Jeong、Geon-Woo Kim、Soojeong Kim 和 Byung-Gon Chun。 2022.Orca：基于 Transformer 的生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22)。 USENIX 协会，卡尔斯巴德，加利福尼亚州，521–538。 https://www.usenix.org/conference/osdi22/presentation/yu [79] 张振凯，泰勒·艾伦，姚范，高兴，葛荣。 2023. 用于盗版的 TunneL：完全逆向工程 GPU TLB，以挑战 NVIDIA MIG 的隔离保证。 2023 年 ACM SIGSAC 计算机和通信安全会议（丹麦哥本哈根）会议记录 (CCS '23)。计算机协会，美国纽约州纽约市，960–974。 https://doi.org/10.1145/ 3576915.3616672 [80] 张振宇，盛颖，周天一，陈天龙，郑连民，蔡瑞思，赵松，田远东，Christopher Ré、Clark Barrett、Zhangyang "Atlas" Wang 和 Beidi Chen。 2023. H2O：用于大型语言模型高效生成推理的重量级预言机。 《神经信息处理系统进展》， A. Oh、T. Naumann、A. Globerson、K. Saenko、M. Hardt 和 S. Levine（编辑），卷。 36. Curran Associates, Inc.，34661–34710。https://proceedings.neurips.cc/paper_files/paper/2023/file/6ceefa7b15572587b78ecfcebb2827f8-Paper-Conference.pdf [81] Lianmin Cheng，Liangsheng Yin，Zhiqiang Xie，Chuyue Sun，Jeff黄、Cody 浩宇、Shiyi Cao、Christos Kozyrakis、Ion Stoica、Joseph E. Gonzalez、Clark Barrett 和 Ying Shen。 2024.SGLang：结构化语言模型程序的高效执行。 arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104 [82] Yinminzhong、Shengyu Liu、Junda Chen、Jianbo Hu、Yibo Zhu、Xu-anzhe Liu、Xin Jin 和 Hao Zhang。 2024.DistServe：分解预填充和解码以实现 Goodput 优化的大型语言模型服务。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。 USENIX 协会，加利福尼亚州圣克拉拉，193-210。 https://www.usenix.org/conference/osdi24/presentation/zhong- yinmin [83] Kan Zhu, Yilong Zhao, Liangyu Zhu, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gau, Qingyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori, Chien- Yu Lin, Stephanie Wang, Arvind克里希那穆蒂和巴里斯·卡西克奇。 2024.NanoFlow：迈向服务吞吐量的最佳大型语言模型。 arXiv:2408.12757 [cs.DC] https://arxiv.org/abs/2408。 12757

A Artifact Appendix

A.1 Abstract

vAttention is a simpler, portable and performant alternative to PagedAttention for dynamic memory management in LLM serving systems. This artifact contains instructions to install vAttention and scripts to reproduce key results of our paper (Figures 2, 3, 4, 6, 7, 8, 9, 11). The scripts are configured to run three large language models: Yi-6B, Llama-3-8B and Yi-34B.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Dynamic memory management for serving LLMs.
- **Run-time environment:** CUDA 12.1, Python 3.10, PyTorch 2.3.0.
- **Hardware:** Two NVLink connected NVIDIA A100 GPUs with 80GB memory each.
- **Disk space required:** 200GB.
- **Time needed to prepare workflow:** 1 hour.
- **Time needed to complete experiments:** 1-2 days.
- **Publicly available?:** Yes
- **Archived DOI?:** https://doi.org/10.5281/zenodo.14048692

A.3 Description

A.3.1 How to access. Clone the artifact as follows:

```
$ git clone https://github.com/microsoft/vattention
```

A.3.2 Hardware dependencies. Running Yi-6B requires one NVIDIA A100 GPU with 80GB memory while the other two models require two NVLink-connected A100 GPUs with 80GB memory each.

A.3.3 Software dependencies. Requires PyTorch 2.3.0 and CUDA 12.1 (later CUDA versions may or may not work).

A.3.4 Expected runtime. Figures 2, 3, and 11 should take a few minutes each. Figures 4, 7 should take about 1 hour each. Figure 6 requires approximately 3 hours. Figures 8 and 9 may consume more than 12 hours each.

A.4 Installation

Move to the directory containing artifact scripts, create and activate a conda environment, then install the artifact as follows:

```
$ cd vattention/scripts/artifact_asplos25/
$ conda create -n vattn python=3.10
$ conda activate vattn
$ (vattn) ./install.sh
```

`vattention/scripts/artifact_asplos25/README.md` file provides detailed installation instructions.

A.5 Alternate Setup: Docker Image

We also provide a docker image for vAttention with all its dependencies pre-installed. To access the docker image, you need to have Docker and NVIDIA Docker installed on your system. You can then launch the docker container and navigate to the folder containing vAttention artifact, as follows:

```
$ docker run --gpus all -it \
  -p 8181:8181 --rm --ipc=host --cap-add=SYS_ADMIN \
  rnp1910/vattention:asplos_25_pytorch_run
$ cd /workspace/vattention/scripts/artifact_asplos25
```

Then follow the experiment workflow detailed in A.7

A.6 Accessing Llama-3-8B

Accessing Llama-3-8B requires logging into huggingface with the user's private token (`HF_TOKEN` below). Login as follows before any experiment:

```
$ (vattn) huggingface-cli login --token HF_TOKEN
```

A.7 Experiment workflow

You can launch all experiments at once or individually as:

```
$ (vattn) ./run_all.sh
          OR
$ (vattn) ./run_figure_2.sh
$ (vattn) ./run_figure_3.sh
```

A.8 Evaluation and expected results

The raw output logs and the final plots will be redirected to `./logs.` and `./plots/` subdirectories within the main artifact directory `vattention/scripts/artifact_asplos25/`. The plots are in the same format as the paper and can be compared directly. The expected result is that non-paged or vAttention based configurations would perform better than the PagedAttention counterparts, especially at longer context lengths. However, the exact numbers may differ from the paper depending on GPUs and software libraries installed on the experiment system.

A.9 Experiment customization

Reproducing Figures 8 and 9 of the paper requires running a large number of requests and hence these experiments can take more than 24 hours to complete. To reduce the experimental time, we have configured their scripts to run with a smaller number of requests (100 each) by default. If you want to run the full trace, execute these scripts with the `--full` argument, i.e., `./run_figure_8.sh --full` and `./run_figure_9.sh --full`. If two NVLink connected GPUs aren't available, update run scripts to avoid running Llama-3-8B and Yi-34B. You can also configure Llama-3-8B to run on a single GPU by setting `--model_tensor_parallel_degree` to 1 in the `./helpers/common.sh` file. Note that such changes would likely impact the absolute performance numbers considerably but vAttention is still expected to perform better than PagedAttention.

神器附录

A.1 摘要

vAttention 是 PagedAttention 的更简单、可移植且高性能的替代方案，用于 LLM 服务系统中的动态内存管理。该工件包含安装 vAttention 和脚本的说明，以重现我们论文的关键结果（图 2、3、4、6、7、8、9、11）。这些脚本配置为运行三种大型语言模型：Yi-6B、Llama-3-8B 和 Yi-34B。

A.2 工件检查表（元信息）

- 算法：为大语言模型的动态内存管理。
- 运行时环境：CUDA 12.1、Python 3.10、PyTorch 2.3.0 。
- 硬件：两个 NVLink 连接的 NVIDIA A100 GPU，每个具有 80GB 内存。
- 所需磁盘空间：200GB。
- 准备工作流程所需时间：1 小时。
- 完成实验所需时间：1-2天。
- 是否公开？： 是
- 存档 DOI？： https://doi.org/10.5281/zenodo.14048692

A.3 描述

A.3.1 如何访问。按如下方式克隆工件：

```
$ git 克隆 https://github.com/microsoft/vattention
```

A.3.2 硬件依赖性。运行 Yi-6B 需要一个具有 80GB 内存的 NVIDIA A100 GPU，而其他两种型号则需要两个 NVLink 连接的 A100 GPU，每个 GPU 具有 80GB 内存。

A.3.3 软件依赖性。需要 PyTorch 2.3.0 和 CUDA 12.1（更高版本的 CUDA 可能工作也可能不工作）。

A.3.4 预期运行时间。图 2、图 3 和图 11 各需要几分钟时间。图 4、图 7 各需要大约 1 小时。图 6 大约需要 3 小时。图 8 和图 9 可能分别花费 12 小时以上。

A.4 安装

移动到包含工件脚本的目录，创建并激活 conda 环境，然后安装工件，如下所示：

```
$ cd vattention/scripts/artifact_asplos25/
$ conda create -n vattn python=3.10
$ conda activate vattn
$ (vattn) ./install.sh
```

vattention/scripts/artifact_asplos25/README.md 文件提供了详细的安装说明。

A.5 替代设置：Docker 映像

我们还为 vAttention 提供了一个 docker 镜像，并预安装了所有依赖项。要访问 docker 镜像，您需要在您的系统上安装 Docker 和 NVIDIA Docker。然后，您可以启动 docker 容器并导航到包含 vAttention 工件的文件夹，如下所示：

```
$ docker run --gpus all -it \
  -p 8181:8181 --rm --ipc=host --cap-add=SYS_ADMIN \
  rnp1910/vattention:asplos_25_pytorch_run
$ cd /workspace/vattention/scripts/artifact_asplos25
```

然后按照 A.7 中详细说明的实验流程进行操作

A.6 访问 Llama-3-8B

访问 Llama-3-8B 需要使用用户的私人令牌（下面的 HF _TOKEN）登录 Huggingface。进行任何实验之前请按如下方式登录：

```
$ (vattn) Huggingface-cli 登录 --token HF_TOKEN
```

A.7 实验流程

您可以一次或单独启动所有实验，如下所示：

```
$ (vattn) ./run_all.sh 或 $ (vattn) .
/run_figure_2.sh $(vattn) ./run_fi
gure_3.sh
```

A.8 评价和预期结果

原始输出日志和最终绘图将被重定向到 ./logs。和主工件目录 vattention/scripts/artifact_asplos25/ 中的 ./plots/ 子目录。这些图的格式与论文相同，可以直接比较。预期的结果是，非分页或基于 vAttention 的配置将比 PagedAttention 对应的配置表现得更好，尤其是在较长的上下文长度下。然而，具体数字可能与论文有所不同，具体取决于实验系统上安装的 GPU 和软件库。

A.9 实验定制

重现本文的图 8 和图 9 需要运行大量请求，因此这些实验可能需要 24 小时以上才能完成。为了减少实验时间，我们将他们的脚本配置为默认运行较少数量的请求（每个 100 个）。如果要运行完整跟踪，请使用 --full 参数执行这些脚本，即 ./run_figure_8.sh --full 和 ./run_figure_9.sh --full。如果两个 NVLink 连接的 GPU 不可用，请更新运行脚本以避免运行 Llama-3-8B 和 Yi-34B。您还可以通过在 ./helpers/common.sh 文件中将 --model_tensor_parallel_ Degree 设置为 1，将 Llama-3-8B 配置为在单个 GPU 上运行。请注意，此类更改可能会极大地影响绝对性能数据，但 vAttention 仍有望比 PagedAttention 表现更好。